

Ingegneria del software

[Appunti di Piai Luca]

Academic Year 2022-2023

Indice

1	Introduzione	4
2	I processi software	4
2.1	Paradigmi di processo	5
2.2	Progettazione ed implementazione	7
2.3	Convalida del software	8
3	Agile	9
3.1	Pratiche di XP	9
3.2	Tecniche di sviluppo agli	10
3.3	Gestione agile della progettazione	12
3.4	Scalabilità dei metodi agili	13
4	Ingegneria dei requisiti	13
4.1	Tipi di requisiti	14
4.2	Processi di ingegneria dei requisiti	15
4.3	Modifica dei requisiti	18
5	Progettazione architetturale	18
5.1	Viste architetturali	18
5.2	Architectural patterns	19
5.2.1	Model-View-Controller	20
5.2.2	Architettura a strati	21
5.2.3	Architettura repository	22
5.2.4	Architettura client-server	22
5.2.5	Architettura pipe-and-filter	23
6	Modellazione dei sistemi	24
6.1	Modelli contestuali	25
6.2	Modelli di interazione	26
6.3	Modelli strutturali	27
6.4	Modelli comportamentali	31
7	Progettazione e implementazione	33
7.1	Progettazione orientata agli oggetti	33
8	GRASP Patterns	40
8.1	Progettazione guidata dalle responsabilità	40
8.2	I GRASP patterns	40
9	GoF Design Patterns	48
9.1	Il catalogo dei pattern	49
9.2	I GoF pattern nel dettaglio	51
10	Test del software	53
10.1	Stadi di test	53

1 Introduzione

Al giorno d'oggi il software ha assunto un ruolo centrale nella società, si trova ormai in qualunque ambito. Per chi lavora in questo settore, un aspetto cruciale riguarda la progettazione di software, in questo corso impareremo delle tecniche utili per poter progettare software di qualità. Ma che cosa si intende con software di qualità? Elenchiamo le caratteristiche da rispettare

- accettabilità: deve essere accettabile per il tipo di utenti che poi si dovranno interfacciare.
- affidabile, sicuro: deve garantire la protezione di dati sensibili e non causare danni (economici o fisici);
- efficiente: non deve richiedere eccessive risorse (tempo e denaro);
- mantenibile: deve potersi evolvere in maniera agevole.

Per riuscire a rispettare questi vincoli dobbiamo attuare un approccio sistematico. Le attività fondamentali da seguire per la creazione di un prodotto sono:

1. specifiche del software: definizioni funzionalità e vincoli;
2. sviluppo del software;
3. convalida del software: appurare che il software rispetti le richieste e i vincoli imposti;
4. evoluzione del software: per soddisfare i cambiamenti dei requisiti in corso d'opera.

Un aspetto fondamentale che influisce sul software è la diversità. I software devono funzionare su diversi dispositivi (PC, dispositivi mobili), inoltre è necessario che software nuovo venga integrato con i vecchi sistemi già in uso.

2 I processi software

Definiamo il processo software come l'insieme delle attività che portano alla creazione di un prodotto software. In generale non esiste un'unica ricetta, può dipendere dal tipo di software, dalle richieste del cliente e dalle persone che scrivono il software.

Possiamo in ogni caso individuare quattro attività fondamentali di un processo software:

1. *specifica del software*: definizione delle funzionalità del software e fissati vincoli operativi;
2. *sviluppo del software*: produzione di un software che soddisfi i requisiti specificati;
3. *convalida del software*: per garantire che risponda a quanto richiesto dal cliente;
4. *evoluzione del software*: evoluzione per soddisfare le necessità del cliente e i cambiamenti dei requisiti di utente.

La descrizione dei processi deve includere:

1. le persone coinvolte;
2. cosa viene prodotto;
3. le condizioni che influiscono sulla sequenza dell'attività.

Lo sviluppo software è un'attività professionale, risulta quindi fondamentale la **pianificazione**. Possiamo individuare due tecniche di pianificazione:

- processi **guidati dai piani**: tutte le attività sono pianificate in anticipo. Il vantaggio sta nel fatto che l'avanzamento viene misurato secondo quanto previsto dal piano;
- processi **agili**: la pianificazione è incrementale. Il vantaggio sta nel fatto che è più facile modificare il processo per adeguarsi ad eventuali modifiche dei requisiti.

In genere si usa un compromesso.

2.1 Paradigmi di processo

I paradigmi di processo sono dei modelli di processo molto generici, descrizioni astratte dei processi software. Con modello software intendiamo una rappresentazione semplificata di un processo software, si tratta di una rappresentazione da un particolare punto di vista che fornisce informazioni parziali sul processo. Analizziamo alcuni modelli:

Modello a cascata Si tratta di un esempio di processo guidato da un piano. In questo modello le attività di processo fondamentali vengono descritte come fasi (stadi) del processo. Il nome deriva dal fatto che le fasi si susseguono una dopo l'altra.

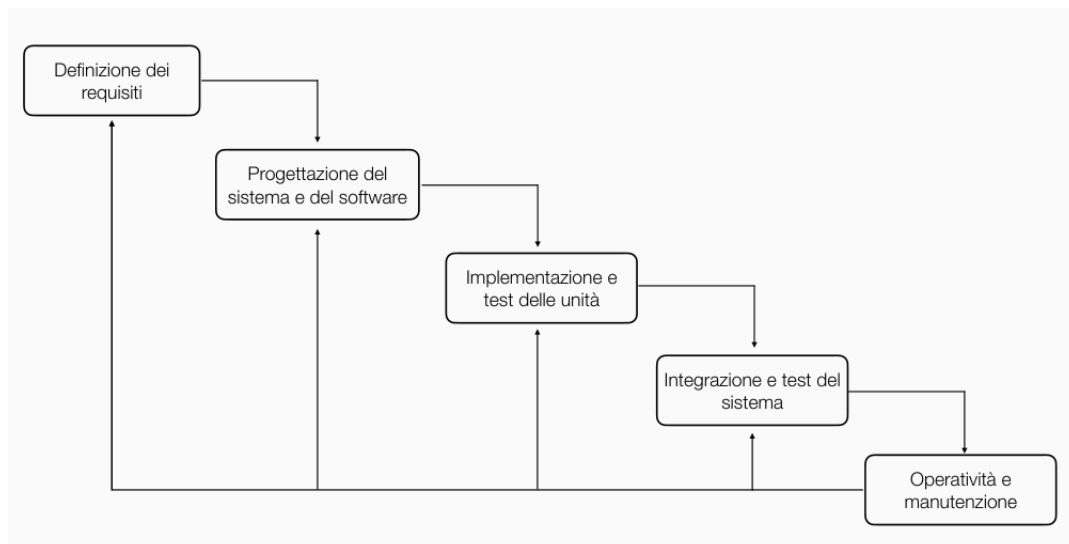


Figura 1: Modello a cascata

- *analisi e definizione dei requisiti*: definizione di servizi, vincoli, e obiettivi del sistema mediante incontri con utenti, i requisiti vengono definiti in dettaglio, diventano specifiche di sistema;
- *progettazione del sistema e del software*: suddivisione dei requisiti sul sistema hardware o software, consente di definire l'architettura del sistema;
- *implementazione e test delle unità*: realizzazione del progetto software come insieme di programmi o unità, test delle unità per verificare che soddisfino le specifiche;
- *integrazione e test del sistema*: unità e programmi integrati in un sistema completo, test con il cliente. Se il test viene superato, il prodotto viene consegnato.

- *operatività e manutenzione*: il sistema è installato e messo in funzione. In questa fase si correggono eventuali errori, si implementano miglioramenti ed eventualmente si aggiungono funzionalità se emergono nuovi requisiti.

In questo modello, dopo ogni fase, viene prodotto uno o più documenti che devono essere approvati. Una fase non dovrebbe partire se la fase precedente non è stata approvata.

Nota che dall'ultima fase si può risalire ad una delle fasi precedenti, ciò avviene in caso di eventuali scoperte di errori, omissioni di requisiti o per la necessità di nuove funzionalità.

Questo tipo di modello è adatto per:

- *sistemi integrati*: software che deve interfacciarsi con sistemi hardware;
- *sistemi critici*;
- *grandi sistemi software*: sistemi sviluppati da più società, che necessitano delle specifiche complete per sviluppare in modo.

Sviluppo incrementale Si parte sviluppando un'implementazione iniziale, viene esposta agli utenti, successivamente si perfeziona tale implementazione tramite molteplici versioni, fino ad ottenere la versione finale del sistema. Riflette il modo di trovare soluzioni per un generico problema: raramente si trova la soluzione al primo tentativo, si procede mediante un insieme di passaggi e si torna indietro in caso di errore.

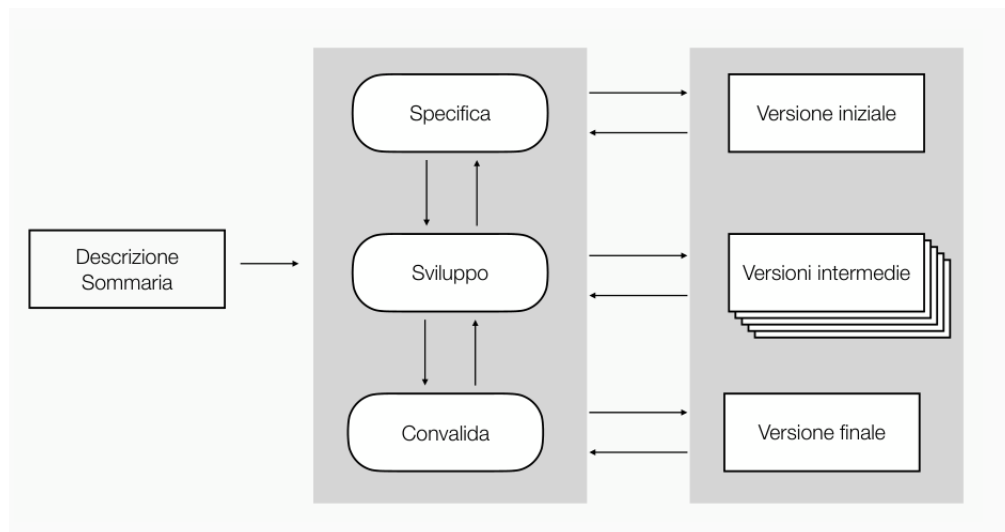


Figura 2: Sviluppo incrementale

La pianificazione può essere *plan-drive*, quindi tutti gli incrementi vanno stabiliti anticipatamente, oppure può essere agile, in questo caso vengono stabiliti solo gli incrementi iniziali, incrementi successivi dipendono dallo stato di avanzamento del lavoro e dalle priorità.

I vantaggi in questo caso sono: i costi ridotti (rispetto al modello a cascata); cliente può valutare se il sistema basato su primi incrementi soddisfa i requisiti.

Questo tipo di approccio si utilizza:

- quando è probabile che i requisiti cambino nel corso del progetto;
- in caso siano coinvolte più sistemi aziendali e più prodotti software da progettare.

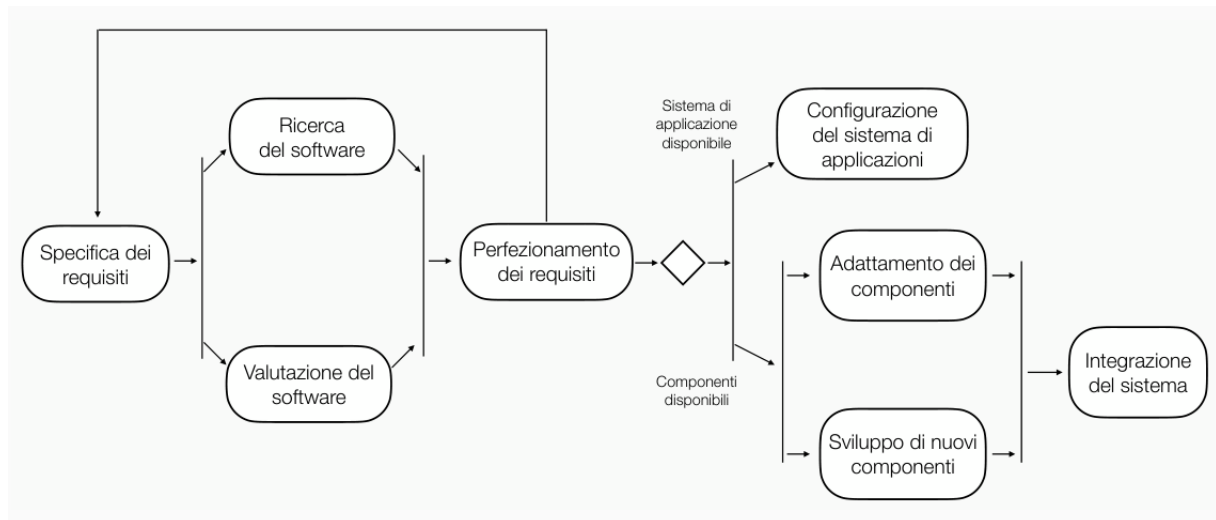


Figura 3: Integrazione e configurazione

Integrazione e configurazione Si basa sul riutilizzo del software esistente (con eventuali modifiche) e successiva implementazione.

Analizziamo le varie fasi di processo:

- *specifica dei requisiti*: requisiti iniziali del sistema (breve descrizione);
- *ricerca e valutazione del software*: svolta sulla base dei requisiti iniziali;
- *perfezionamento dei requisiti*: in base a informazioni su applicazioni e componenti trovati. Se modifiche alle specifiche non sono possibili, si torna al passo precedente;
- *configurazione del sistema delle applicazioni*: se il sistema di applicazioni è disponibile, configurazione per creazione di un nuovo sistema;
- *adattamento ed integrazione dei componenti*.

I vantaggi di questo modello sono: la ridotta quantità di software sviluppato; il ridotto costo; la riduzione dei tempi necessari. Come svantaggi: compromessi nei requisiti; perdita di controllo nell'evoluzione del sistema, i componenti non sono sotto il controllo degli sviluppatori.

Per quanto riguarda i modelli di processo per **grandi sistemi**, spesso è utile combinare le migliori caratteristiche dei paradigmi che abbiamo analizzato. Alcuni sottoinsiemi del processo possono essere implementabili con approcci differenti, per esempio: possiamo utilizzare il modello a cascata o integrazione per parti già chiare e definite nella fase iniziale (definizione/descrizione requisiti); si può utilizzare un approccio incrementale per parti difficili da specificare preventivamente.

2.2 Progettazione ed implementazione

L'obiettivo è quello di convertire le specifiche in un sistema eseguibile. Nel progetto del software bisogna descrivere la struttura del software, dei modelli, delle strutture dati e delle interfacce tra i componenti del sistema o tra gli algoritmi usati.

Spesso le interfacce software si interfacciano con altri sistemi, per esempio: sistema operativo; database; middleware; sistemi di applicazioni. Tali sistemi costituiscono la **piattaforma software**, le informazioni sulla piattaforma vanno incluse nell'input di progettazione.

Per quanto riguarda le *attività di progettazione*:

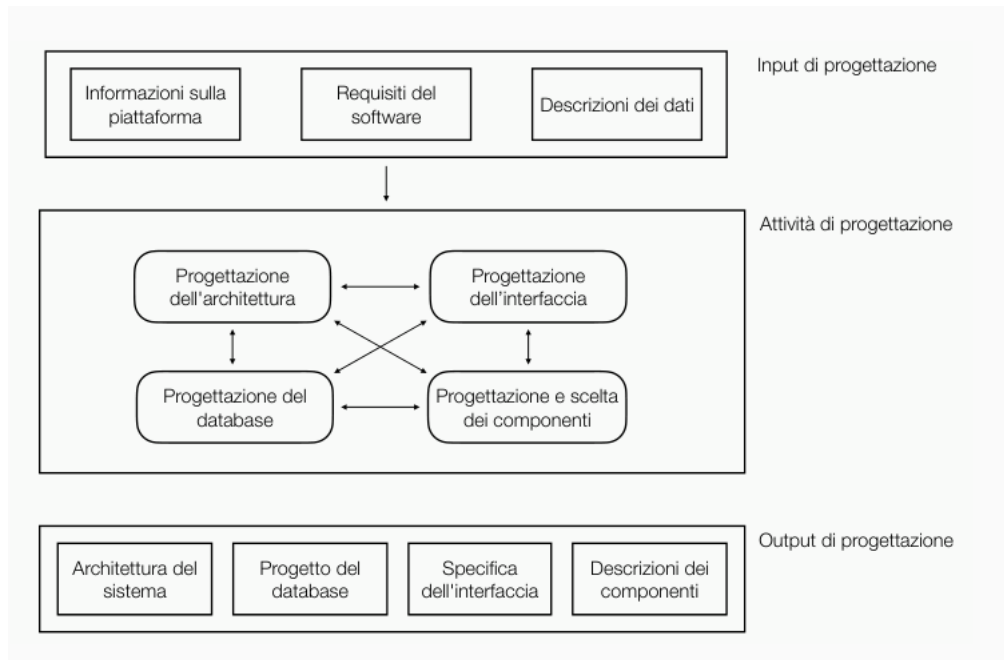


Figura 4: Sviluppo incrementale

- **progettazione dell'architettura:** identificazione della struttura complessiva del sistema, dei moduli (componenti principali), delle relazioni e della distribuzione dei moduli;
- **progettazione del database:** identificazione dei dati del sistema, della loro struttura e della loro rappresentazione in un database;
- **progettazione dell'interfaccia:** definizione non ambigua dell'interfaccia tra componenti. I componenti possono essere progettati separatamente;
- **progettazione e scelta dei componenti:** ricerca dei componenti, progettazione di eventuali modifiche necessarie per componenti idonei, progettazione di nuovi componenti;

Per quanto riguarda gli *output di progettazione* possiamo seguire il seguente schema:

- documenti dettagliati per sistemi critici;
- diagrammi di progettazione per approcci guidati da modelli;
- rappresentati nel codice del programma per approcci agli.

L'**implementazione**, di norma, è un'attività individuale che dipende dal programmatore. Il dubbio che può venire è: da dove si parte? Parto dalle componenti che ho compreso o che conosco, o prima quelli che ho compreso meno? In generale non c'è una risposta, dipende dal progetto. Talvolta si parte da uno scheletro, dal risultato della fase di progettazione.

2.3 Convalida del software

L'obiettivo è che il software sia conforme alle specifiche e rispetti le aspettative del cliente. La tecnica principale di convalida consiste nel testare dei programmi con dati simulati.

Possiamo dividere in tre stadi il processo di test:

1. **test componenti:** componenti testati separatamente dagli sviluppatori, ogni sviluppatore crea i propri test;

2. **test del sistema:** integrazione dei componenti nel sistema completo, verifica di possibili errori dovuti a interazione tra componenti;
3. **test del cliente:** stadio precedente alla messa in produzione, testato dal cliente con i dati reali, mette in evidenza errori o omissioni nella definizione dei requisiti o problemi con i requisiti se non soddisfano le necessità del cliente.

Può essere utile il rilascio di un *beta test*. In questo modo i clienti possono testare il software per un certo periodo di tempo, possono sottoporlo a casi reali e in caso di riscontri negativi possono segnalare i problemi riscontrati agli sviluppatori.

DA FINIRE

3 Agile

Le idee alla base di Agile vengono sviluppate contemporaneamente da persone diverse negli anni '90. Uno degli approcci più significativi è quello dell'**Extreme Programming**.

Nell'Extreme Programming i requisiti vengono espressi come scenari o come *user stories*, che a loro volta vengono divise in una serie di *task* da implementare. I programmatori lavorano a coppie e prima di mettersi a scrivere codice, scrivono i test per ogni task. Per poter rilasciare una *release*, tutti i test devono essere eseguiti con successo.

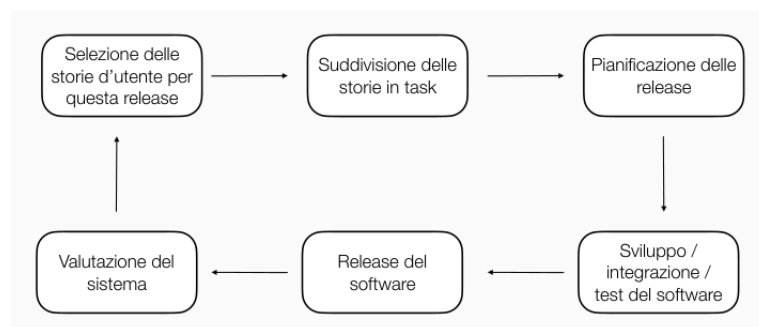


Figura 5: Ciclo di rilascio della programmazione estrema

3.1 Pratiche di XP

Si basano su alcuni punti fondamentali:

- **pianificazione incrementale:** i requisiti si basano su semplici scenari che vengono registrati su story card. Tali storie vengono suddivise in task di sviluppo;
- **progettazione semplice:** viene progettato solo il necessario a soddisfare i requisiti correnti;
- **piccole release:** inizialmente vengono sviluppate il minimo numero di funzionalità utili, successivamente si effettuano release frequenti, ciascuna delle quali aggiunge alla release iniziale nuove funzionalità in modo incrementale;
- **integrazione continua:** appena un task viene concluso, se ha superato tutti i test, viene integrato nel sistema;
- **cliente on-site:** è a tutti gli effetti un membro del team, ha la responsabilità di consegnare requisiti del sistema a team XP;

- **programmazione a coppie:** utile per la verifica reciproca del lavoro e per migliorare la qualità;
- **proprietà collettiva:** conseguenza della pair programming; coppie di sviluppatori lavorano su tutte le aree del sistema, in questo modo non si vengono a creare esperti in aree isolate. Così facendo, tutti sono responsabili di tutto il codice e tutti hanno la possibilità di apportare modifiche al sistema;
- **refactoring:** non appena si trovano possibili miglioramenti al codice viene fatto il refactoring, in questo modo il codice è semplice e mantenibile;
- **sviluppo con test iniziali:** spesso si utilizza un ambiente automatico di test delle unità, tutti i test vengono scritti prima di implementare la funzionalità;
- **ritmo sostenibile:** non sono accettabili grandi ritardi, questo perché i ritardi comportano a una riduzione della qualità del codice e della produttività a medio termine.

3.2 Tecniche di sviluppo agili

Prevedono quattro punti:

1. user stories;
2. refactoring;
3. sviluppo con test iniziali;
4. programmazione a coppie;

User stories Sostanzialmente descrivono i requisiti, gli scenari d'uso in cui potrebbe trovarsi un utente del sistema. Il cliente collabora con il team di sviluppo, insieme producono le *story card*: raccolgono le esigenze dell'utente. Successivamente il team svilupperà queste story card e rilascerà funzionalità corrispondenti nella release successiva.

Risulta necessario interagire con il cliente per definire le priorità delle user stories (basata sull'utilità della funzionalità per l'azienda).

I vantaggi delle user stories: sono efficaci in quanto consentono all'utente di relazionarsi in maniera più semplice rispetto che ai documenti dei requisiti, inoltre sono efficaci nell'aumentare il coinvolgimento dell'utente. Per quanto riguarda gli svantaggi: spesso sono incomplete (in termini di definizione dei requisiti), inoltre utenti esperti (del sistema) possono omettere requisiti importanti perché li danno per scontati.

Refactoring L'ingegneria del software tradizionale prevede di *progettare il cambiamento*, l'XP scarta questo principio: ritenuto uno sforzo inutile, alcune delle modifiche previste potrebbero non avvenire mai. L'XP gestisce questo aspetto *rifattorizzando* costantemente il codice. L'idea di base è la seguente: quando i membri del team vedono che il codice può essere rifattorizzato, realizzano i miglioramenti immediatamente. Lo scopo è quello di evitare modifiche locali, che tendono a deteriorare la struttura del sistema e di evitare modifiche future in quanto sono sempre più difficili da realizzare.

Il refactoring non è una pratica sempre possibile, può essere rinviato per dedicare più tempo allo sviluppo di nuove funzionalità, inoltre alcune modifiche non sono possibili mediante refactoring perché richiedono una modifica dell'architettura.

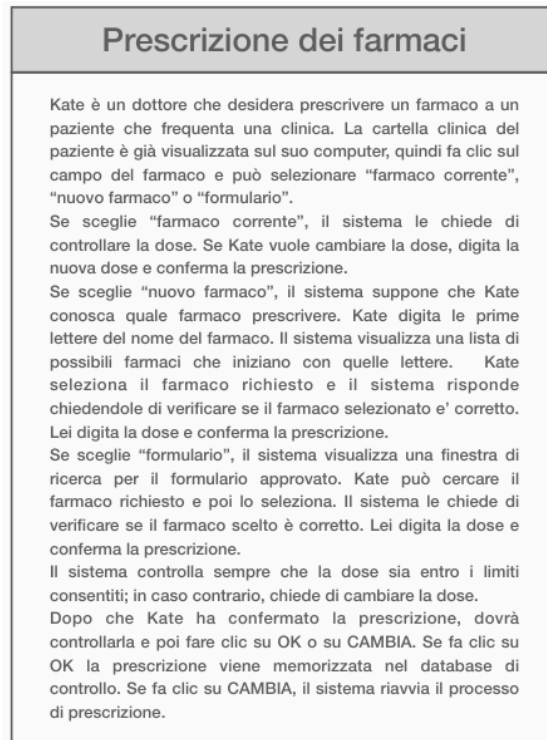


Figura 6: Esempio di story card per Mentcare

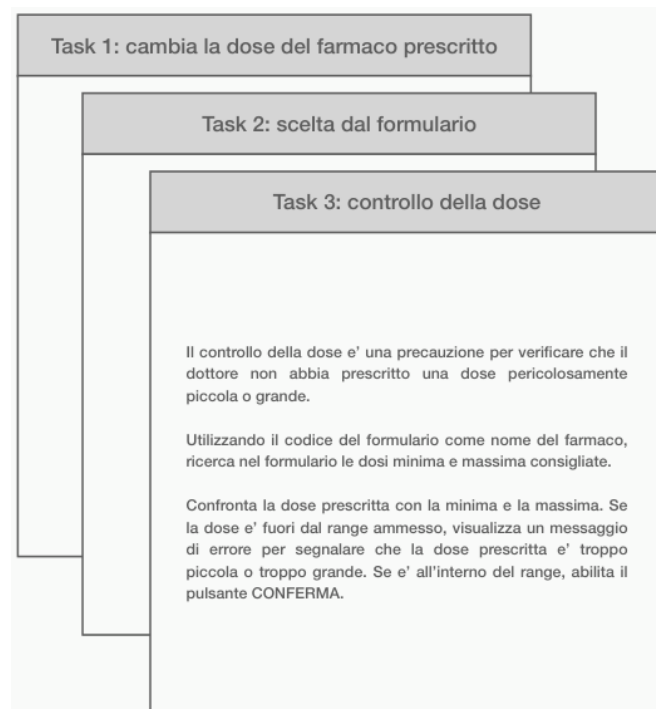


Figura 7: Da story card a task

Sviluppo con test iniziali I test vengono effettuati in modo molto diverso rispetto allo sviluppo guidato da piani. Come abbiamo anticipato, vengono scritti i test, poi il codice. Il vantaggio è che risulta più semplice scoprire eventuali problemi in fase di sviluppo, inoltre in

questo modo non è possibile posticipare lo sviluppo dei test.

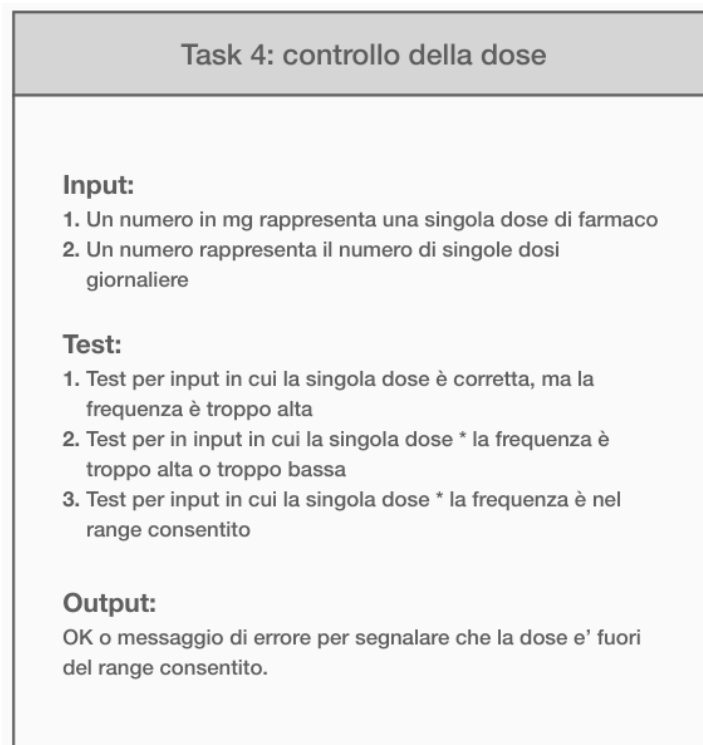


Figura 8: Esempio scrittura di un test

Per ogni task da implementare viene definito uno o più test. Solitamente per svolgere i test si sfruttano delle componenti automatiche in grado di simulare l'input e controllare che il risultato soddisfi la specifica.

Programmazione a coppie I vantaggi sono i seguenti: il team è responsabile di problemi del codice e la risoluzione è una responsabilità collettiva (*egoless programming*); il processo di revisione avviene in maniera informale, talvolta risulta meno efficace dell'ispezione del codice, ma è meno costosa; incentiva il refactoring.

La domanda che può sorgere è: in questo modo si produce la metà? Studi formali danno risultati contrastanti: la produzione è in linea con quella individuale, vengono evitate false partenze e ci sono inferiori rielaborazioni del codice (grazie al confronto); con programmatori più esperti si ha una perdita di produttività anche se la qualità risulta più alta. Sicuramente un importante beneficio: riduce il rischio di fallimento nel caso di abbandono di uno dei membri.

3.3 Gestione agile della progettazione

La gestione del tempo e delle risorse è in fattore fondamentale anche nello sviluppo agile. Una soluzione al riguardo è il **metodo Scrum**: si tratta di un framework per organizzare progetti, fornisce visibilità esterna su ciò che sta accadendo. Questo metodo, non essendo focalizzato su specifiche pratiche di sviluppo, può essere integrato con metodi esistenti nelle società.

Vediamo i concetti essenziali di Scrum:

- **team di sviluppo:** gruppo di sviluppatori software con organizzazione autonoma, non dovrebbe avere più di 7 persone. Il team è responsabile dello sviluppo software e dei documenti essenziali di progettazione;
- **sprint:** iterazione dello sviluppo, solitamente dura da 2 a 4 settimane;
- **incremento di un prodotto potenzialmente rilasciabile:** incremento del software che è consegnato da uno sprint. Tale incremento deve essere "potenzialmente" rilasciabile.
- **product backlog:** lista degli elementi di cui si deve occupare il team di Scrum. Comprende per esempio: storie utente, descrizioni task, definizione architettura, etc.
- **product owner:** si tratta di un individuo o di un piccolo gruppo che ha il compito di identificare caratteristiche o requisiti del prodotto, stabilire le priorità, rivedere continuamente il product backlog per garantire che il progetto continui a soddisfare i requisiti critici;
- **Scrum:** si tratta di una riunione giornaliera del team. Lo scopo è quello di esaminare l'avanzamento del lavoro e di stabilire le priorità per il lavoro da eseguire quel giorno;
- **scrum master:** deve garantire che il processo Scrum sia eseguito, deve guidare i team nell'uso efficiente di Scrum, deve garantire il team non venga sviato da interferenze esterne, non deve essere considerato un project manager;
- **velocità:** stima della quantità di lavoro del product backlog che il team può svolgere in un singolo sprint.

3.4 Scalabilità dei metodi agili

Da come abbiamo descritto i metodi agili, risulta evidente che sono più adatti a sviluppatori di piccoli team, per sviluppare prodotti di piccole/medie dimensioni. La domanda che ci poniamo è: come utilizzare metodi agili in grandi società, per grandi progetti?

DA FINIRE (vedi se conviene)

4 Ingegneria dei requisiti

I requisiti forniscono una descrizione dei servizi che un sistema deve offrire. In generale, l'ingegneria dei requisiti, si occupa della ricerca, analisi, documentazione e verifica dei servizi e vincoli. Il termine **requisito** non viene utilizzato in modo coerente: talvolta come descrizione formale di un servizio e dei suoi vincoli; talvolta come descrizione astratta. Questo perché le società che devono dare in appalto software, in una prima fase descrivono le funzionalità e i vincoli in modo astratto, dando flessibilità in termini di soluzioni al problema; successivamente, dopo l'assegnazione dell'appalto, le società scrivono un documento molto più dettagliato.

Possiamo definire due livelli di descrizione dei requisiti:

- **requisiti d'utente:** specificano quali servizi il sistema deve fornire. Vengono specificati utilizzando un linguaggio naturale e/o diagrammi;
- **requisiti di sistema:** descrivono dettagliatamente le funzioni, i servizi e i vincoli. Dovrebbero definire esattamente cosa deve essere implementato.

La fase iniziale dell'ingegneria dei requisiti dovrebbe fornire una vista generale di cosa il sistema deve fare, questo al fine di decidere se procedere. Si parla di studio di fattibilità. Lo studio dovrebbe rispondere a tre domande chiave:

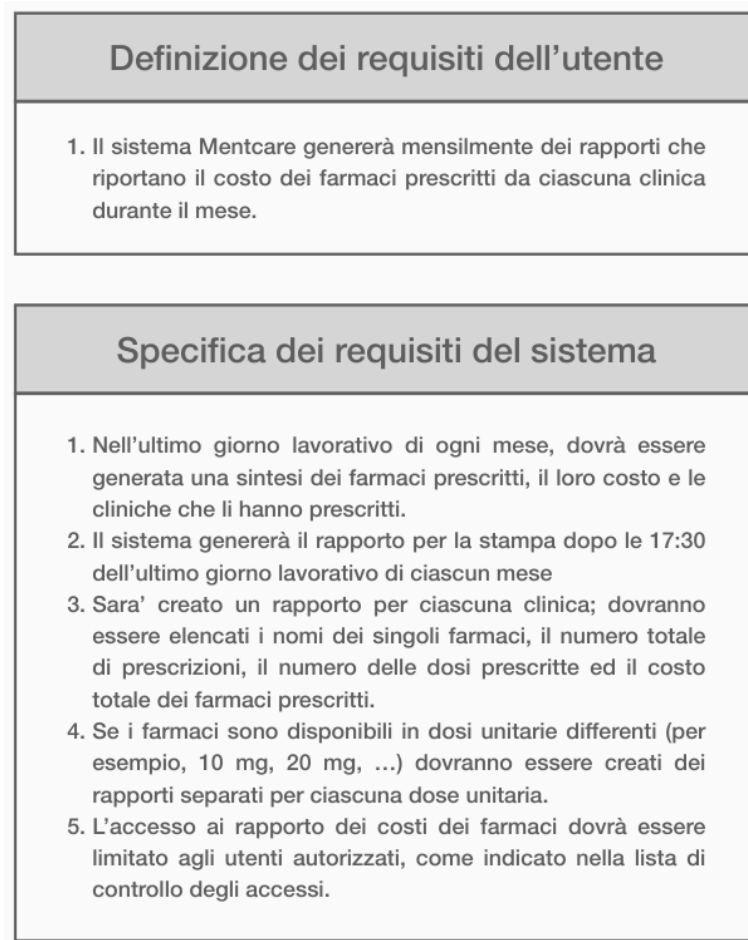


Figura 9: Esempio di descrizione dei requisiti

1. il sistema contribuirà al raggiungimento degli obiettivi generali dell'organizzazione?
2. Il sistema può essere implementato usando la tecnologia attuale nel rispetto dei costi e dei tempo prefissati?
3. Il sistema può essere integrato con altri sistemi già utilizzati?

4.1 Tipi di requisiti

Dividiamo i requisiti in due categorie: **requisiti funzionali**, questi definiscono i servizi che il sistema deve fornire, come deve reagire a particolari input e a particolari situazioni; **requisiti non funzionali**, questi specificano i vincoli sulle funzioni/servizi offerti dal sistema, vincoli temprali sul processi di sviluppo o legati a normative. I requisiti funzionali descrivono ciò che il sistema deve fare, se vengono espressi come requisiti d'utente devono essere scritti in linguaggio naturale. Mentre i requisiti non funzionali sono utili agli sviluppatori, questi estendono i requisiti d'utente e specificano dettagliatamente aspetti come per esempio: funzioni; input e output; eccezioni. I requisiti non funzionali non sono legati alle funzioni, ma specificano o limitano alcune caratteristiche, per esempio: l'affidabilità del sistema; i tempi di risposta; l'uso della memoria. In genere sono più critici dei requisiti funzionali in quanto non sono aggirabili: se non vengono soddisfatti il sistema non funziona. Basti pensare per esempio all'affidabilità nei sistemi di controllo degli aerei.

Sottolineiamo il fatto che questo tipo di requisiti può influire sull'intera architettura del sistema: se viene imposto di dover rispettare un certo tempo di risposta, allora ciò implica che bisogna minimizzare le comunicazioni tra i componenti da progettare. In generale, un requisito non funzionale può generare diversi requisiti funzionali o limitare quelli esistenti.

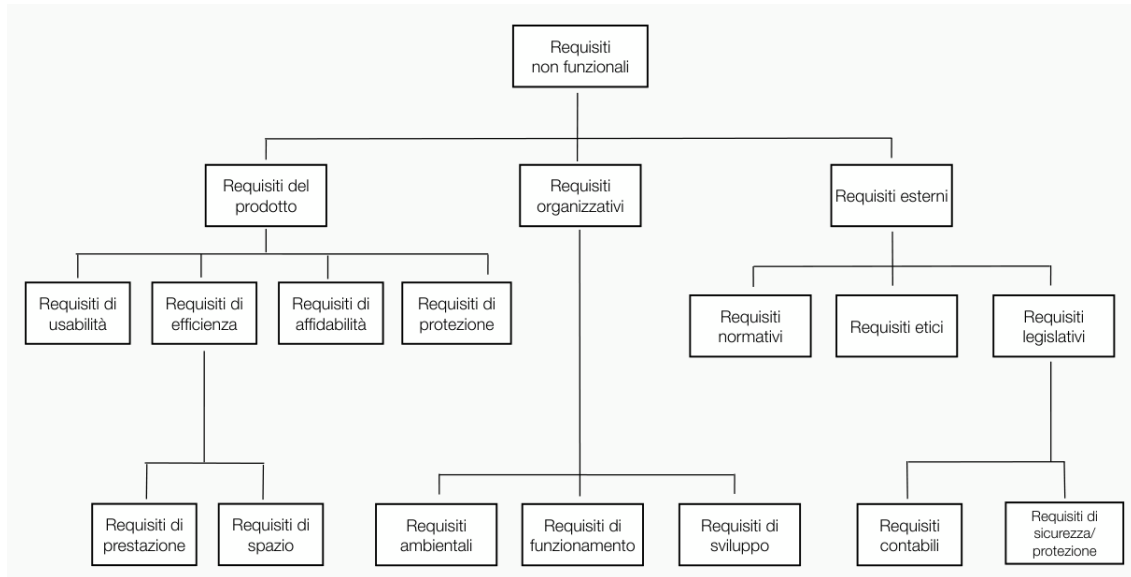


Figura 10: Albero requisiti non funzionali

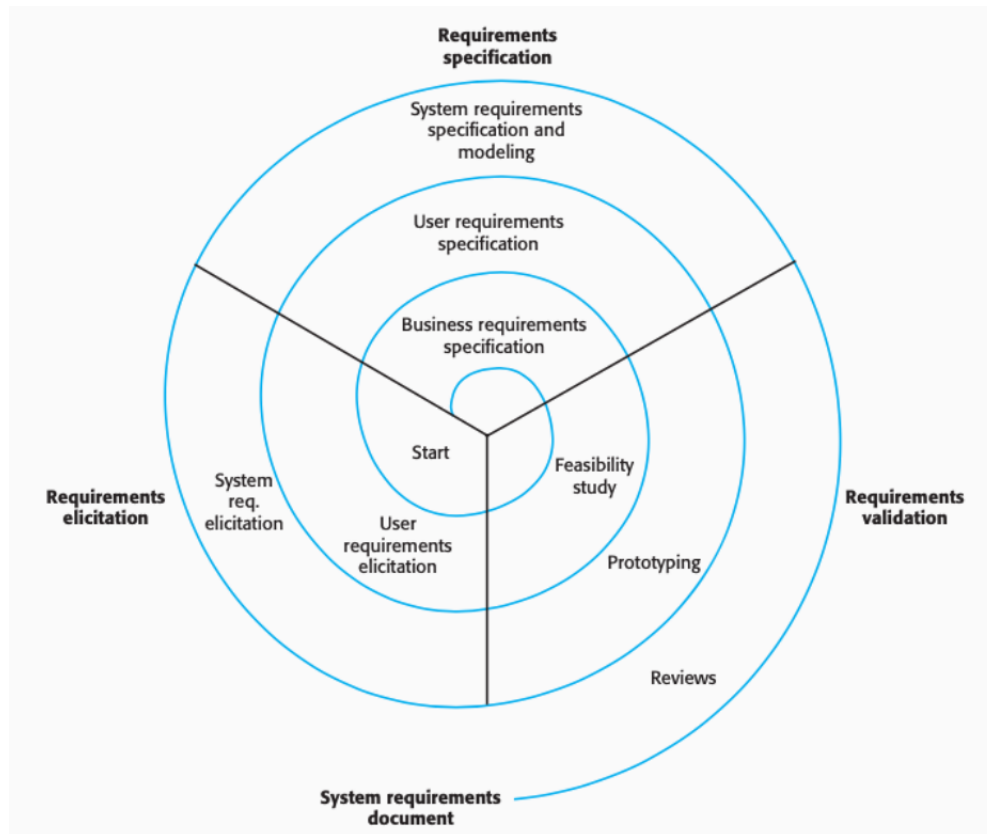
Proprietà	Misura
Velocità	Transazioni elaborate al secondo Tempi di risposta a utenti/eventi Tempo di refresh dello schermo
Dimensione	Megabyte
Facilità d'uso	Tempo di addestramento Numero di maschere d'aiuto
Affidabilità	Tempo medio di malfunzionamento Probabilità di indisponibilità Tasso di malfunzionamenti Disponibilità
Robustezza	Tempo per il riavvio dopo un malfunzionamento Percentuale di eventi che causano malfunzionamenti Probabilità di corruzione dei dati dopo un malfunzionamento

Figura 11: Misure per specificare requisiti non funzionali

A livello pratico è difficile distinguere i requisiti funzionali dai non funzionali, risulta difficile distinguerli nel documento dei requisiti e non possono essere dichiarati separatamente perché altrimenti le relazioni tra di essi risulta difficile da capire.

4.2 Processi di ingegneria dei requisiti

Si tratta di un modello a spirale diviso in tre spicchi. Analizziamoli uno per uno.



Deduzione dei requisiti L'obiettivo è quello di comprendere il lavoro degli stakeholder. Bisogna raccogliere informazioni su: dominio di applicazione; attività; servizi e caratteristiche del sistema; prestazioni richieste; vincoli hardware. Gli aspetti problematici di questa fase sono che, spesso, gli stakeholder sanno descrivere cosa vogliono in termini molto generici e spesso fanno richieste irrealizzabili; tendono inoltre ad esprimere i requisiti mediante le loro conoscenze di dominio e tendono ad esprimere requisiti diversi in modi differenti. Possiamo riassumere la deduzione dei requisiti con un ciclo (Figura 12).

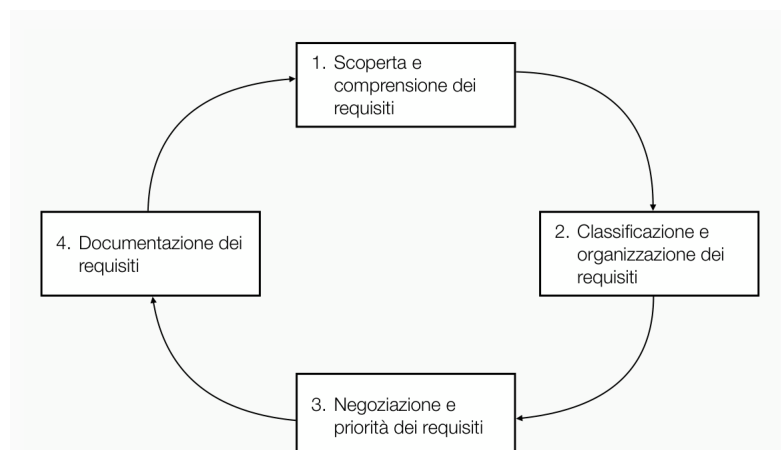


Figura 12: Deduzione dei requisiti

Si tratta di un processo iterativo e continuo; se sono presenti molteplici stakeholder bisogna dividerli in gruppi, ciascuno costituisce un punto di vista e bisogna dare voce a ciascuno.

Durante questa fase, per dedurre i requisiti si utilizzano due tecniche: le interviste e/o l'etnografia. Le interviste consistono in una serie di domande predefinite (o non) agli stakeholder, mentre l'etnografia è una tecnica di osservazione attuata da un analista.

Specifica dei requisiti In questa fase bisogna scrivere i requisiti d'utente e di sistema in un documento. I requisiti dovrebbero essere non ambigui, facili da capire, completi e coerenti. Il problema è che spesso non rispecchiano queste caratteristiche: contengono incoerenze e conflitti intrinseci.

Differenziamo le specifiche in base al tipo dei requisiti: i *requisiti d'utente* dovrebbero includere requisiti funzionali e non, dovrebbero essere leggibili e comprensibili dagli utenti, quindi non è adatto un gergo tecnico, non dovrebbero includere aspetti tecnici e architetturali e infine dovrebbero essere focalizzate sul comportamento "esterno" del sistema; mentre i *requisiti di sistema* rappresentano un'espansione dei requisiti d'utente, le specifiche venono utilizzate dagli ingegneri del software come base di partenza per la progettazione, le specifiche devono spiegare come il sistema deve comportarsi per realizzare i requisiti d'utente, in genere dovrebbero riguardare requisiti esterni e vincoli ma non aspetti di progettazione.

Notazione	Descrizione
Fraasi nel linguaggio naturale	Requisiti scritti utilizzando frasi numerate nel linguaggio naturale; ogni frase deve esprimere un requisito
Linguaggio naturale strutturato	Requisiti scritti nel linguaggio naturale secondo una forma o un modello standard; ogni campo fornisce informazioni su un aspetto del requisito
Notazioni grafiche	Modelli grafici, corredati da annotazioni di testo, usati per definire requisiti funzionali del sistema. Utilizzo di casi d'uso e diagrammi di sequenza in UML
Specifiche matematiche	Notazioni basate su concetti matematiche come insiemi o macchine a stati finiti. Riducono l'ambiguità ma sono di difficile comprensione per i clienti e non sono in grado di verificarne la correttezza; spesso riluttanti ad accettarli come parte del contratto.

Figura 13: Notazioni per descrivere i requisiti di sistema

Covalida dei requisiti Bisogna verificare se i requisiti definiscono il sistema realmente voluto dal cliente. Questa fase è fondamentale, permette di evitare costi di rilavorazione nelle fasi successive. Riassumiamo le tipologie di controllo da effettuare durante il processo di convalida:

- *controlli di validità*: verifica se requisiti riflettono reali esigenze del cliente;
- *controlli di consistenza*: no conflitti, contraddizioni, duplicati;
- *controlli di completezza*: il documento deve contenere tutte le funzioni e vincoli;
- *controlli di realismo*: verifica che i requisiti possano essere realmente implementabili.

4.3 Modifica dei requisiti

Capita spesso che i requisiti per grandi sistemi software cambino: ne emergono di nuovi e altri vengono scartati. La modifica è probabile soprattutto se i clienti (acquirenti) del sistema non sono gli utilizzatori diretti oppure se sono presenti diversi (gruppi di) stakeholder che hanno precisato requisiti in conflitto.

La gestione della modifica avviene attraverso le seguenti fasi:

- analisi dei problemi e specifica delle modifiche;
- analisi e stima dei costi delle modifiche da apportare;
- implementazione delle modifiche sul sistema.

Se le modifiche sono urgenti, bisogna ricordarsi di aggiornare il documento dei requisiti.

5 Progettazione architetturale

La progettazione architetturale si occupa dell'organizzazione di un sistema software e della progettazione della sua struttura complessiva, si tratta del collegamento tra progettazione ed ingegneria dei requisiti. L'output di tale fase è un modello architetturale che descrive l'organizzazione del sistema in funzione dei componenti e di come questi comunicano.

L'**architettura software** non è il software (operativo), bensì una rappresentazione che consente di analizzare l'efficacia del progetto e a considerare alternative architetturali (in fase di progettazione).

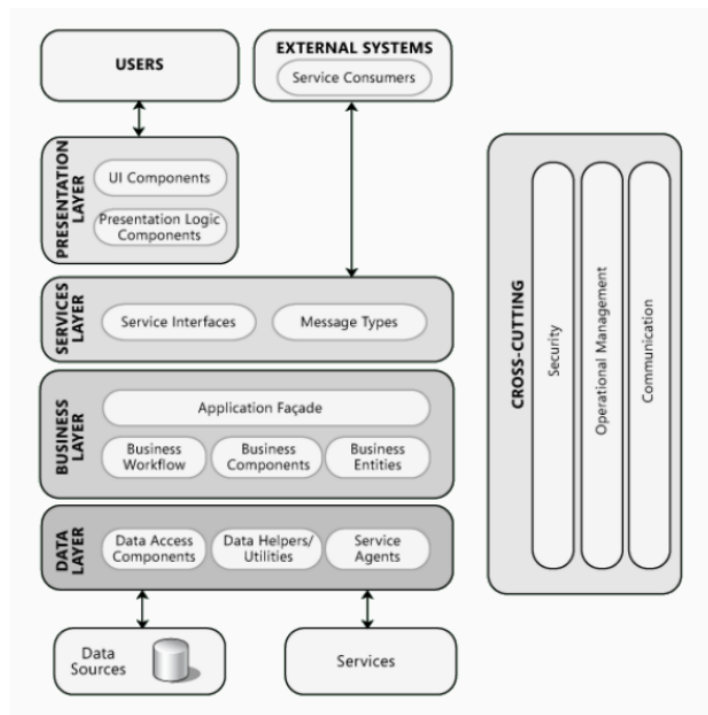


Figura 14: Esempio di architettura software

5.1 Viste architetturali

La domanda che ci si pone è: quali sono le viste o prospettive utili quando si progetta e si documenta l'architettura di un sistema? Quali notazioni si utilizzano per descrivere i modelli

architetture? Una **vista** rappresenta un insieme di elementi che compongono la struttura e ne mette in evidenza le relazioni. Siccome non è possibile rappresentare tutte le informazioni relative ad una architettura in un singolo diagramma, si utilizzano più viste dell'architettura. Non esistono delle viste standard da utilizzare, un possibile approccio è il **modello 4+1 viste dell'architettura**: ci sono quattro viste fondamentali collegabili tramite scenari o casi d'uso comune.

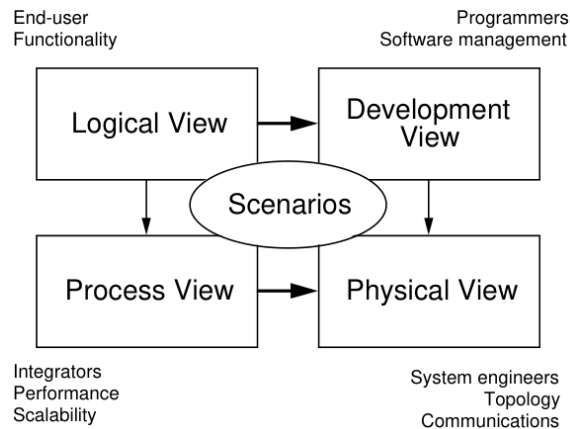


Figura 15: Esempio di architettura software

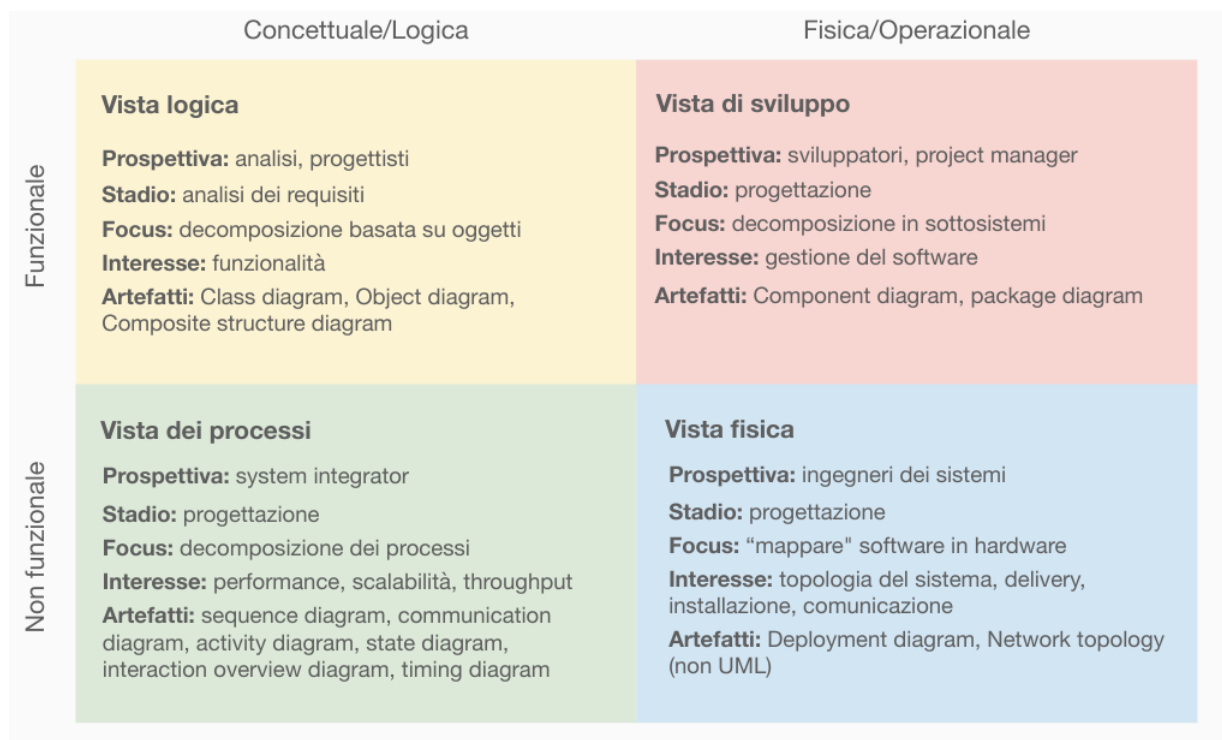


Figura 16: 4+1 viste

5.2 Architectural patterns

I pattern possiamo descriverli come una sorta di "déjà vu": riconoscere la somiglianza tra problemi già affrontati e saper adattare soluzioni già trovate. Perché utilizzare i pattern? Sono

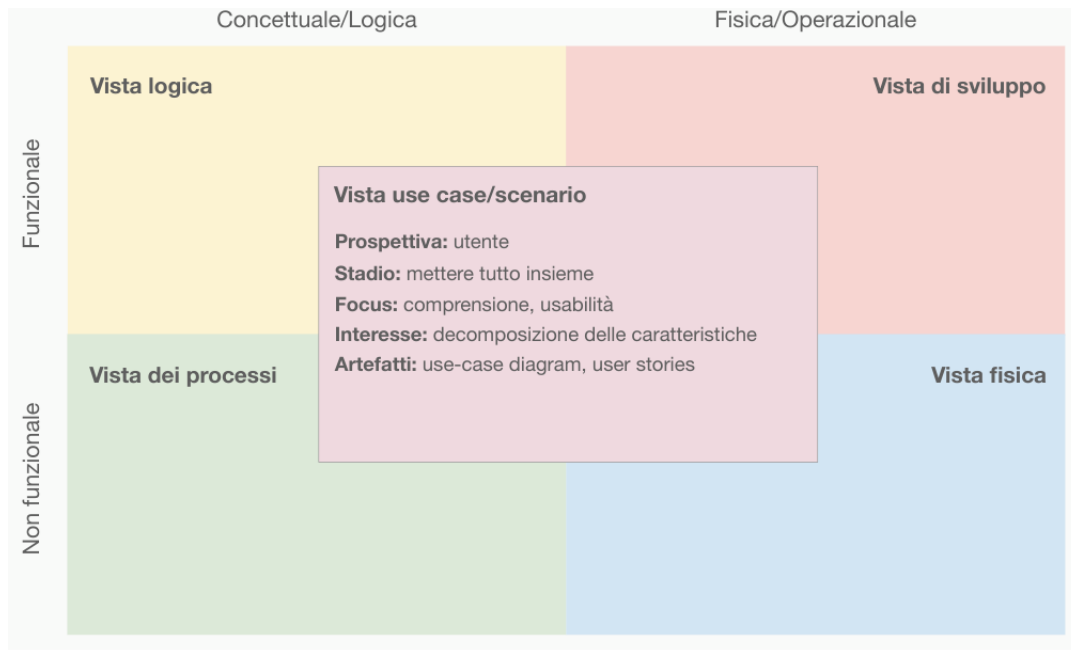


Figura 17: 4+1 viste

molto utili in quanto aiutano a scegliere tra diverse alternative e facilitano la gestione della documentazione e della manutenzione. Per rappresentare i pattern utilizziamo descrizioni stilizzate.

Esistono diversi schemi (o stili) architetturali, ci concentriamo su 5 in particolare:

- Model-View-Controller (MVC Pattern);
- Architettura a strati (Layered Pattern);
- Architettura repository (Repository Pattern);
- Architettura client-server (Client-server Pattern);
- Architettura pipe-and-filter (Pipe-and-filter Pattern).

5.2.1 Model-View-Controller

Si tratta di una triade di classi usate in Smalltalk-80 per costruire interfacce grafiche. Si basa su tre tipologie di oggetti:

- Model: oggetto dell'applicazione;
- View: rappresentazione a video dell'oggetto;
- Controller: definizione delle reazioni dell'interfaccia grafica all'input dell'utente.

Gli oggetti model e View sono disaccoppiati, per comunicare utilizzano un protocollo di sottoscrizione/modifica: la View deve garantire che l'aspetto rifletta lo stato del Model; se lo stato del Model cambia, le View vengono notificate e aggiornate di conseguenza.

1	Descrizione:
2	Presentazione e interazione separate dai dati del sistema. Il sistema
3	e' strutturato in tre componenti logiche che interagiscono tra loro. Il

componente Model gestisce i dati del sistema e le operazioni associate. Il componente View definisce e gestisce il modo in cui i dati sono presentati all'utente. Il componente Controller gestisce l'interazione degli utenti e passa queste interazioni ai componenti Model e View.

Esempio:

Un sistema di applicazioni basate sul Web organizzate secondo lo schema MVC.

Quando si usa:

Si usa quando ci sono piu' modi di visualizzare e interagire con i dati; si usa anche quando non sono noti i requisiti futuri di interazione e presentazione dei dati.

Vantaggi:

Consente ai dati di cambiare indipendentemente dalla loro rappresentazione e viceversa. Supporta la presentazione degli stessi dati in modi differenti, con le modifiche fatte in una rappresentazione mostrate in tutte le altre.

Svantaggi:

Potrebbe richiedere altro codice o un codice complesso quando il modello dei dati e le interazioni sono semplici.

5.2.2 Architettura a strati

Viene anche detta n-tier architecture pattern. Si tratta di un'architettura divisa in molteplici strati che si scambiano informazioni. Il vantaggio sta nel fatto che: se non vengono modificate le interfacce, gli strati sono completamente intercambiabili; se si cambiano interfacce di uno strato, basta modificare solo gli strati adiacenti.

Descrizione:

Organizza il sistema in vari strati, con funzionalita' associate a ciascuno strato. Uno strato fornisce i servizi allo strato sopra di esso, quindi gli strati di livello piu' basso rappresentano i servizi di base che probabilmente saranno utilizzati in tutto il sistema.

Esempio:

Un modello a strati di un sistema di apprendimento digitale che supporta l'apprendimento di tutti i soggetti nelle scuole

Quando si usa:

Si usa per costruire nuove funzioni per un sistema esistente; quando lo sviluppo e' distribuito fra piu' team, dove ogni team ha la responsabilita' di sviluppare le funzioni di uno strato; quando c'e' una richiesta di protezione su piu' livelli.

Vantaggi:

Consente la sostituzione di interi strati, se l'interfaccia non viene modificata. Le funzioni ridondanti (autenticazione) possono essere fornite in ciascuno strato per aumentare la fidatezza del sistema.

Svantaggi:

Nella pratica spesso e' difficile ottenere una netta separazione fra gli strati, e uno strato di alto livello potrebbe aver bisogno di interagire direttamente con strati di livelli inferiori, anziche' con lo strato immediatamente sotto di esso. Le performance possono essere un problema, a causa dei vari livelli di interpretazione di una richiesta di servizio, essendo questa elaborata in ciascuno strato.

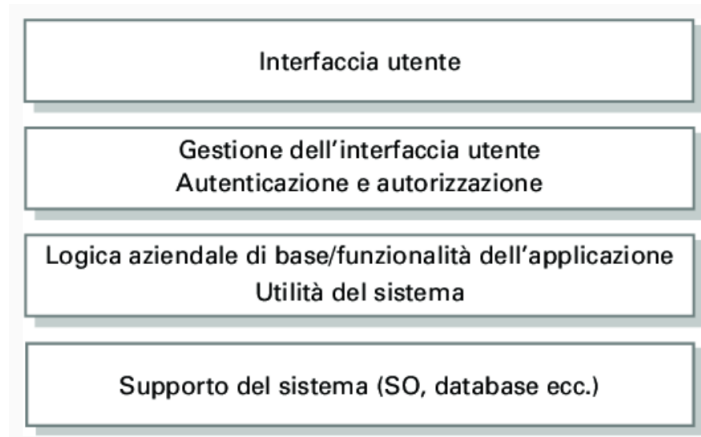


Figura 18: Esempio architettura a strati

5.2.3 Architettura repository

1	Descrizione:
2	Tutti i dati di un sistema vengono gestiti in un database centrale
3	(repository), accessibile da tutti i componenti del sistema. I compo-
4	nenti non interagiscono direttamente, ma attraverso il repository.
5	Esempio:
6	IDE dove i componenti usano un repository di informazioni per la
7	progettazione di un sistema. Ogni strumento software genera infor-
8	mazioni, che poi sono messe a disposizione degli altri strumenti.
9	Quando si usa:
10	Si usa quando nel sistema vengono generati grandi volumi di infor-
11	mazioni che devono essere memorizzate per lungo tempo. Si puo'
12	usare anche nei sistemi guidati dai dati dove l'inclusione dei dati
13	nel repository innesca un'azione o l'utilizzo di uno strumento.
14	Vantaggi:
15	I componenti possono essere indipendenti; un componente non de-
16	ve necessariamente sapere dell'esistenza di un altro componente.
17	Le modifiche fatte da un componente possono essere rese note agli
18	altri componenti. Tutti i dati vengono gestiti in modo coerente in
19	quanto si trovano nello stesso posto.
20	Svantaggi:
21	Il repository e' un punto comune di malfunzionamento. Potrebbero
22	esserne delle inefficienze nell'organizzazione di tutte le comunicazio-
23	ni con il repository. Potrebbe essere difficile distribuire il repository
24	su piu' computer.

5.2.4 Architettura client-server

1	Descrizione:
2	Sistema presentato come un insieme di servizi, ciascuno dei quali e'
3	fornito da un server separato. I client sono utenti di questi servizi e
4	accedono ai server per utilizzare tali servizi.
5	Esempio:
6	Libreria di film e video/DVD organizzata come un sistema client-
7	server.

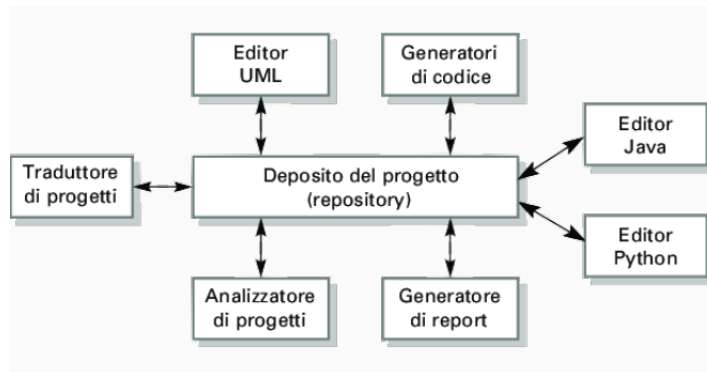


Figura 19: Esempio architettura repository

8 Quando si usa:

9 Si usa quando occorre accedere ai dati di un database da piu' posta-
 10 zioni. Poiche' i server possono essere replicati, lo schema puo' essere
 11 utilizzato anche quando il carico su un sistema e' variabile.

12 Vantaggi:

13 Il principale vantaggio di questo modello e' che i server possono es-
 14 sere distribuiti in una rete. Le funzionalita' piu' comuni (per esempio,
 15 il servizio di stampa) possono essere a disposizione di tutti i client e
 16 non devono essere necessariamente implementate da tutti i servizi.

17 Svantaggi:

18 Ciascun servizio e' un punto comune di malfunzionamento, nel senso
 19 che e' suscettibile di attacchi denial-of-service ed e' soggetto a gua-
 20 sti del server. Le prestazioni possono essere imprevedibili in quanto
 21 dipendono dalla rete e anche dal sistema. Possono nascere proble-
 22 mi di gestione se i server sono di proprieta' di piu' organizzazioni.

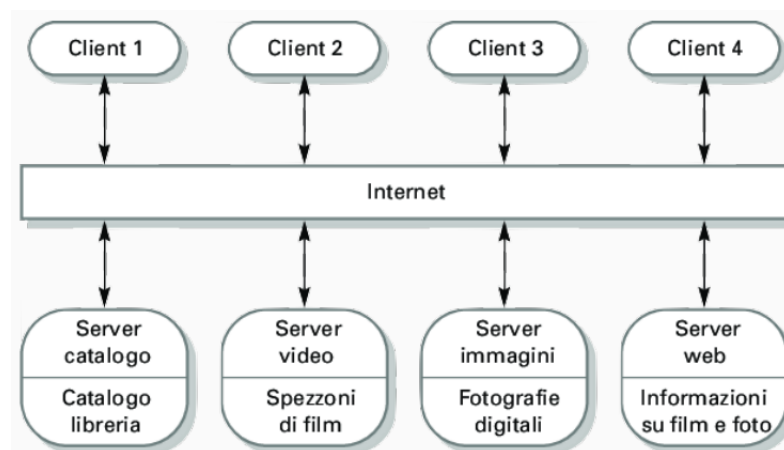


Figura 20: Esempio architettura client-server

5.2.5 Architettura pipe-and-filter

1 Descrizione:

2 L'elaborazione dei dati del sistema e' organizzata in modo che cia-
 3 scun componente di elaborazione (filtro) e' discreto e svolge un par-

4 ticolare tipo di trasformazione dei dati. I dati fluiscono, come in un
 5 tubo (pipe), da un componente all'altro per essere elaborati.
 6 Esempio:
 7 Sistema pipe-and-filter per l'elaborazione di fatture'
 8 Quando si usa
 9 Si usa di solito nelle applicazioni per l'elaborazione dei dati (batch
 10 o basata su transazioni), dove gli input vengono elaborati in fasi
 11 separate per generare i relativi output.
 12 Vantaggi:
 13 E' facile da capire e supporta il riutilizzo delle trasformazioni. Lo stile
 14 del flusso delle operazioni rispecchia la struttura di molti processi
 15 aziendali. La sua evoluzione e' semplice perche' permette di aggiun-
 16 gere agevolmente le nuove trasformazioni. Puo' essere implementa-
 17 to come sistema sequenziale o parallelo.
 18 Svantaggi:
 19 Il formato di trasferimento dei dati deve essere concordato fra le va-
 20 rie trasformazioni. Ciascuna trasformazione deve essere in grado di
 21 leggere gli input e fornire gli output nel formato concordato. Questo
 22 aumenta gli overhead del sistema e potrebbe rendere impossibile
 23 il riutilizzo di quei componenti architetturali che usano strutture di
 24 dati non compatibili con il formato concordato.

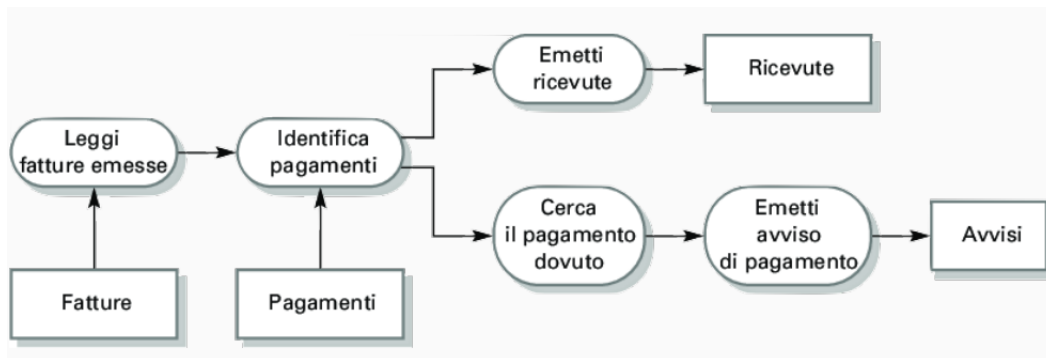


Figura 21: Esempio architettura pipe-and-filter

6 Modellazione dei sistemi

La modellazione è un processo che sviluppa modelli astratti di un sistema. Con modello si intende una particolare vista o prospettiva del sistema, generalmente non comprende tutti i dettagli ma solo alcune caratteristiche. Esistono diverse prospettive da cui si può modellare un sistema:

- prospettiva esterna: contesto o ambiente in cui opera il sistema;
- prospettiva di interazioni: interazioni tra sistema e ambiente o tra componenti del sistema;
- prospettiva strutturale: organizzazione del sistema o struttura dati;
- prospettiva comportamentale: risposte agli eventi.

Per rappresentare le prospettive risulta conveniente sfruttare una notazione grafica, per esempio possiamo utilizzare UML (Unified Modeling Language). UML include 13 tipi di

diagrammi organizzati in due gruppi, generalmente 5 sono sufficienti a rappresentare gli elementi essenziali del sistema:

1. diagrammi di attività: attività coinvolte in un processo o nell'elaborazione dei dati;
2. diagrammi di casi d'uso: interazioni tra un sistema ed il suo ambiente;
3. diagrammi di sequenza: interazioni tra attori e sistema e tra componenti;
4. diagramma di classe: classi del sistema e loro associazioni;
5. diagrammi di stato: come il sistema reagisce agli eventi interni ed esterni.

6.1 Modelli contestuali

Lo scopo è quello di fissare i confini del sistema: specificare cosa fa parte e cosa no. Per poter definire tale modello è importante definire le funzionalità che deve includere il sistema e quali processi e operazioni eseguire nell'ambiente del sistema.

Si nota dalla Figura 22 che sono presenti altri sistemi ma non vengono rappresentate le relazioni tra di essi. Per rappresentare le relazioni utilizziamo i diagrammi di attività UML, come viene riportato nella Figura 23. La notazione che si utilizza per questo tipo di diagrammi è la seguente:

- Inizio processo: cerchio pieno;
- Fine processo: cerchio con all'interno un altro cerchio;
- Attività / sottoprocessi: rettangoli con spigoli arrotondati;
- Flusso di lavoro: frecce;
- Coordinamento attività: barra piena;
- Tutte le attività che portano ad una barra piena vanno concluse prima di procedere;
- Attività che partono da una barra piena possono essere svolte in parallelo;
- Condizione per cui flusso viene eseguito: parentesi quadre vicino alle frecce.

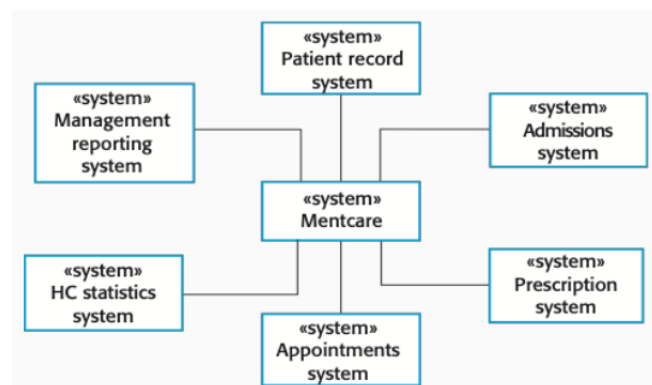


Figura 22: Esempio modello contestuale Mentcare

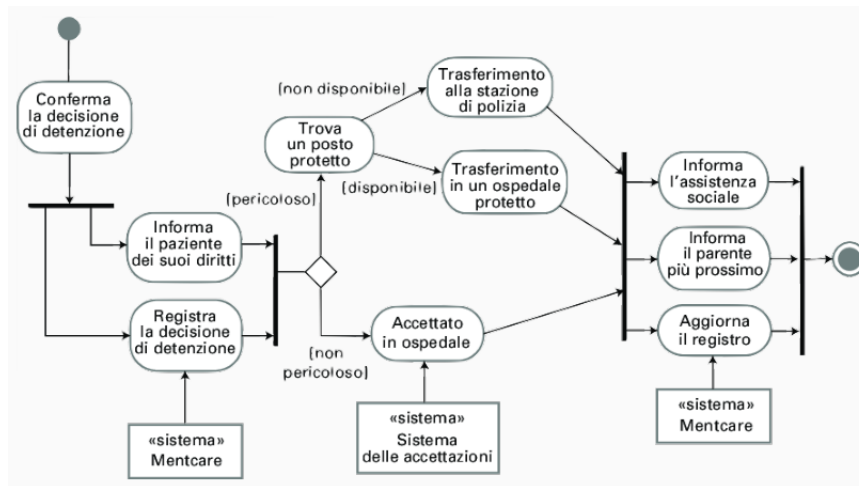


Figura 23: Esempio diagramma di attività Mentcare

6.2 Modelli di interazione

Tutti i modelli hanno un qualche forma di interazione, per esempio: interazione con l'utente; interazione con altri sistemi nello stesso ambiente; interazione tra componenti del sistema. L'interazione deve essere modellata, ciò aiuta ad identificare i requisiti d'utente, i problemi di comunicazione e verificare la fidatezza e le prestazioni richieste.

Esistono due approcci alla modellazione delle interazioni:

- modellazione dei casi d'uso: utile per modellare interazioni tra sistema e/o utenti;
- diagrammi di sequenza: utilizzati per modellare interazioni tra componenti, è possibile anche aggiungere agenti esterni.

I *modelli dei casi d'uso* descrivono ciò che l'utente si aspetta dal sistema per una determinata interazione. La notazione utilizzata: il caso d'uso viene rappresentato da un ellisse; gli attori coinvolti (non per forza umani) vengono rappresentati mediante figura stilizzate.



Il caso d'uso rappresentato riguarda il trasferimento di dati dal sistema Mentcare a un sistema per la gestione dei pazienti. Gli attori coinvolti sono due: l'operatore che trasferisce i dati; il sistema di registrazione dei pazienti. Il diagramma in sé non basta, esso rappresenta una semplice panoramica di un caso d'uso, deve essere integrata anche una descrizione testuale. La descrizione in questo caso potrebbe essere la seguente:

- | | |
|---|--|
| 1 | Attori: |
| 2 | Personale di accettazione, sistema di registrazione dei pazienti (SRP). |
| 3 | Descrizione: |
| 4 | Un addetto all'accettazione può trasferire i dati dal sistema Mentcare a |
| 5 | un database generale di registrazione dei pazienti che è mantenuto da |
| 6 | un'autorità sanitaria. Le informazioni trasferite possono essere |
| 7 | informazioni personali aggiornate (indirizzo, numero di telefono ecc.) |
| 8 | o una sintesi della diagnosi e della cura dei pazienti. |

```

9  Dati:
10  Informazioni personali dei pazienti, sintesi della cura.
11  Stimolo:
12  Comando emesso dal personale di accettazione.
13  Risposta:
14  Conferma che l'SRP e' stato aggiornato.
15  Commenti:
16  L'addetto all'accettazione deve avere le autorizzazioni appropriate per
17  accedere alle informazioni dei pazienti e all'SRP.

```

I diagrammi di sequenza mostrano le interazioni con il sistema durante un particolare caso d'uso. Sono bidimensionali: l'asse verticale è l'asse temporale; l'asse orizzontale indica le relazioni tra i componenti; sempre sull'asse verticale è presente una colonna (chiamata *lifeline*) che indica l'inizio e la fine della vita dell'oggetto.

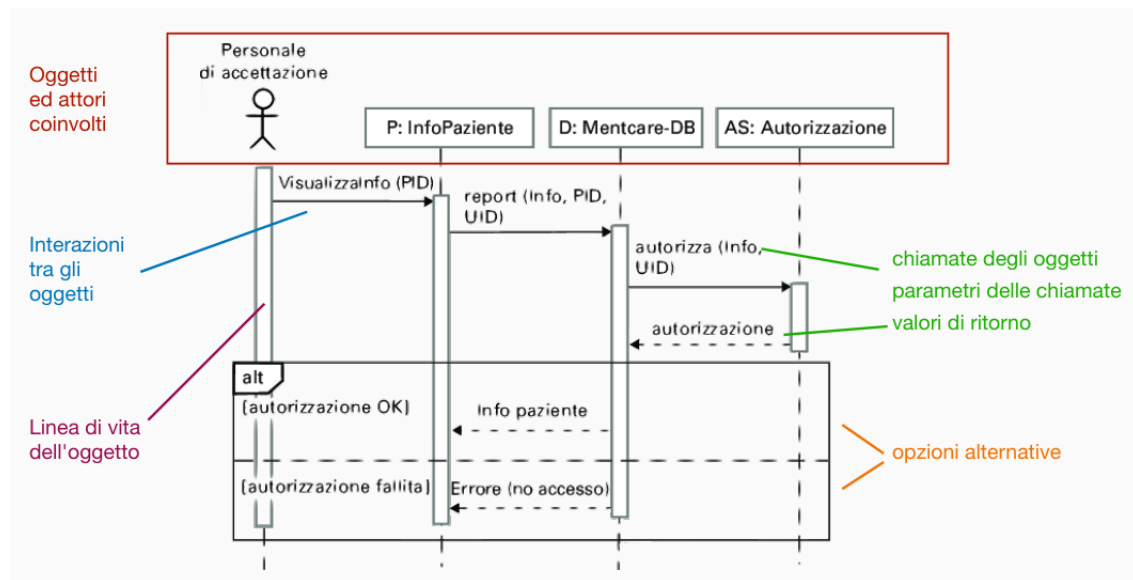
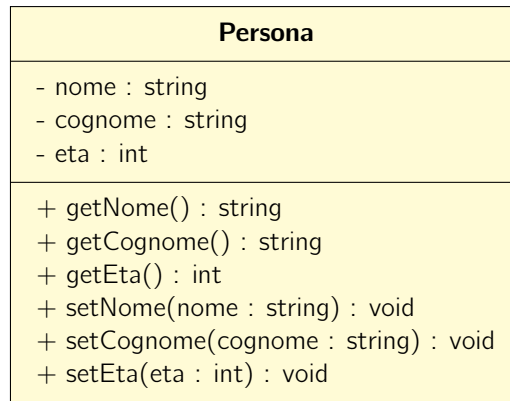


Figura 24: Esempio diagramma di sequenza Mentcare

6.3 Modelli strutturali

Mettono in evidenza l'organizzazione di un sistema in funzione dei suoi componenti e le relazioni che sussistono tra i componenti. Risultano utili quando dobbiamo progettare l'architettura del sistema o quando dobbiamo descriverla. Noi in particolare ci concentreremo sugli oggetti del sistema e le loro relazioni.

Per rappresentare le relazioni tra gli oggetti utilizziamo i diagrammi di classe, questi possono essere rappresentati a diversi livelli di dettaglio che dipendono direttamente dalla fase in cui ci troviamo (es: fase iniziale $\Rightarrow$ pochi dettagli).

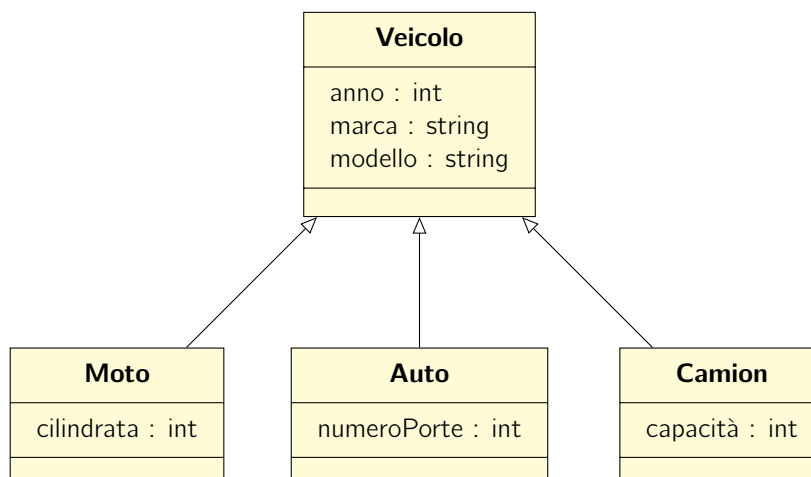


Elenchiamo le caratteristiche dei diagrammi di classe:

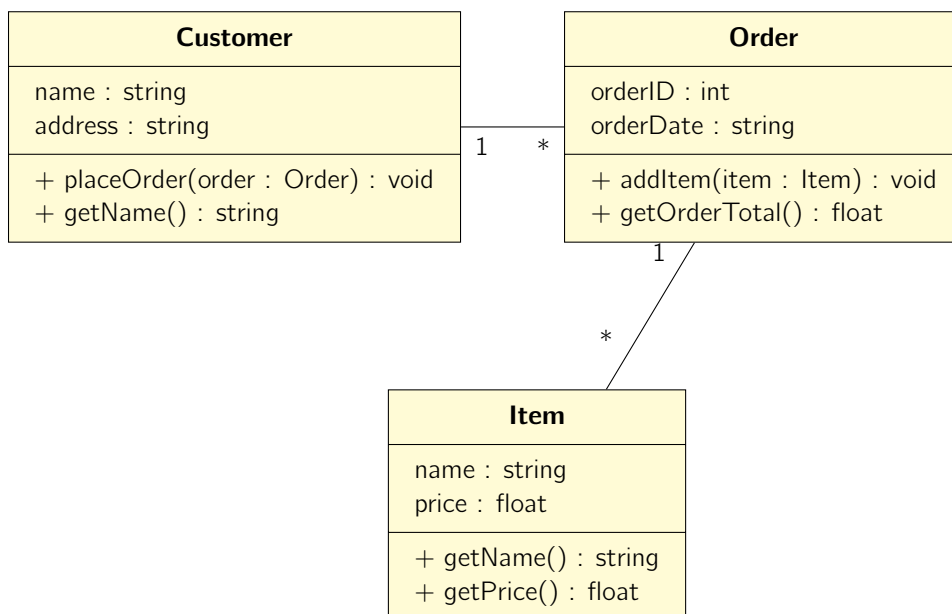
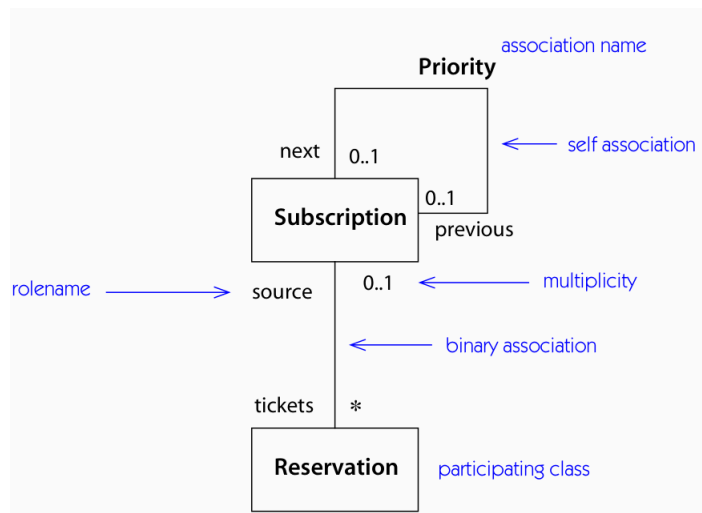
- il nome della classe si trova nel primo compartimento del rettangolo ed è l'unico campo obbligatorio per la rappresentazione grafica;
- gli attributi (es: `name: String`) sono proprietà della classe che descrivono l'oggetto da modellare. Si trovano nel secondo compartimento e la sintassi da utilizzare è `attributeName: Type`. Inoltre possono essere:
 - public se preceduti da +;
 - protected se preceduti da -;
 - derived se preceduti da /;
- le operazioni (metodi/funzioni) descrivono il comportamento della classe e vengono riportati nel 3° compartimento. Le operazioni possono essere specificate riportando il nome, il tipo, i valori di default dei parametri e il return type.

In UML le interconnessioni (logiche o fisiche) tra oggetti sono modellate con relazioni. Esistono tre tipi di relazioni: **generalizzazioni**; **associazioni**; **dipendenze**.

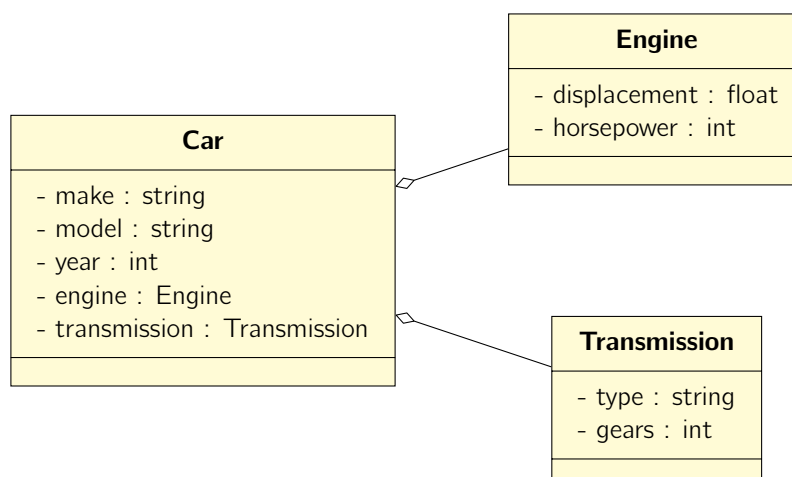
In fase di modellazione di un sistema è utile valutare la generalizzazione, in modo tale da mantenere le informazioni comuni in un solo posto e di conseguenza limitare il numero di modifiche.



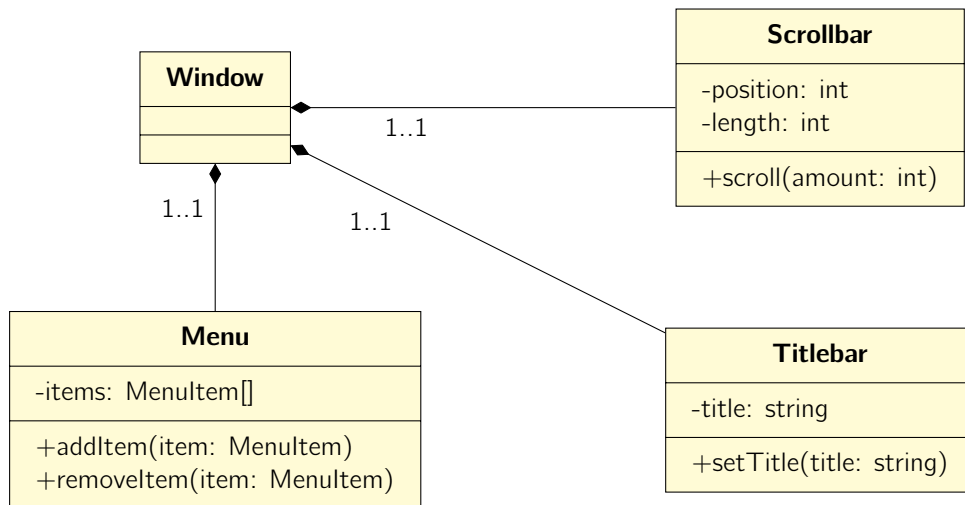
Un'associazione tra due classi viene utilizzata se tra di esse sussiste una relazione strutturale. Le associazioni forniscono le connessioni attraverso le quali oggetti di classi differenti possono interagire. Un esempio di associazione è la molteplicità, questa potrebbe essere per esempio 1 : 3 (uno a tre), oppure 1..* (uno a molti). Possiamo anche specificare un nome all'associazione e possiamo anche specificare il comportamento dell'oggetto nell'associazione (es: il suo ruolo).



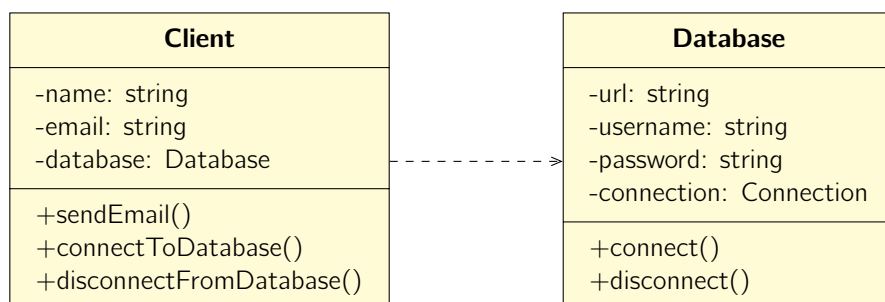
Gli oggetti nel mondo reale possono essere composti da diverse parti, necessitiamo dunque di rappresentare la relazione di *aggregazione*. In questo tipo di relazione le parti costituenti possono esistere indipendentemente.



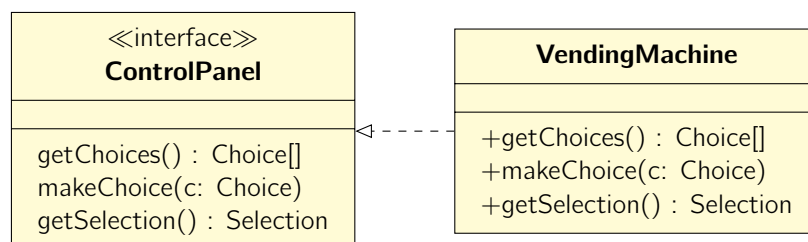
Un'altra relazione molto comune è la *composizione*: indica una relazione di appartenenza più forte della parte rispetto al tutto. In questo caso la parti non sono indipendenti.



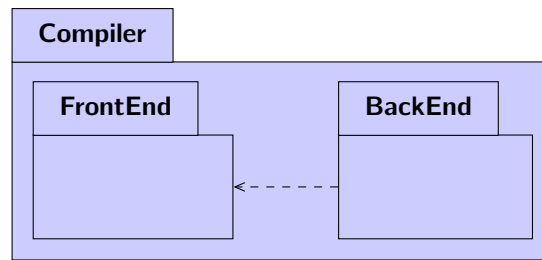
Una dipendenza indica una relazione semantica tra due o più elementi di un modello. Generalizzazione e associazione sono dipendenze per definizione, vengono trattate a parte in quanto hanno una semantica specifica. Il termine dipendenza viene utilizzato per tutte le relazioni che non rientrano in una categoria specifica.



In UML esiste il concetto di *stereotipo* (*stereotype*), si tratta di un'estensione di un costrutto dello UML standard. sostanzialmente ci permette di definire un elemento che ha la stessa forma di un altro ma con un diverso intento. Ciò ci permette di rappresentare in UML le interfacce. Attraverso la relazione di *realizzazione* possiamo specificare che una classe implementa l'interfaccia.



Un ultimo elemento che introduciamo per i diagrammi di classe è il *package*: permette di organizzare altri elementi in gruppi. Può contenere classi, altri package o diagrammi.



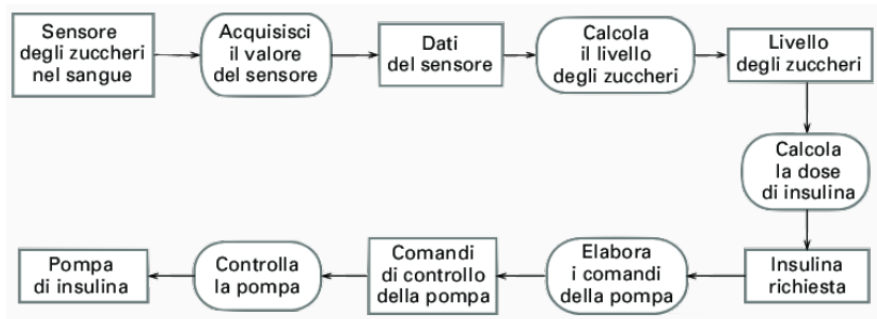
6.4 Modelli comportamentali

Questo tipo di modelli mostrano cosa accade (o si suppone che accada) in risposta ad uno stimolo esterno. Individuiamo due tipologie di stimoli:

- i dati: vengono resi disponibili dati che devono essere elaborati dal sistema, la disponibilità innesca il processo di elaborazione;
- gli eventi: elaborazione del sistema innescata da un evento, gli eventi non sono necessariamente associati ai dati.

In base alla tipologia degli stimoli, distinguiamo i modelli comportamentali in modelli guidati dai dati e guidati dagli eventi.

I modelli guidati dai dati mostrano la sequenza delle azioni coinvolte nell'elaborazione dei dati di input e nella generazione dell'output; sono utili nell'analisi dei requisiti per mostrare tutti i passi dell'elaborazione. Possiamo rappresentare questi modelli tramite i diagrammi di attività UML: i passi elaborazione vengono rappresentati come attività (rettangoli a spigoli arrotondati); i dati vengono rappresentati come oggetti (rettangoli).



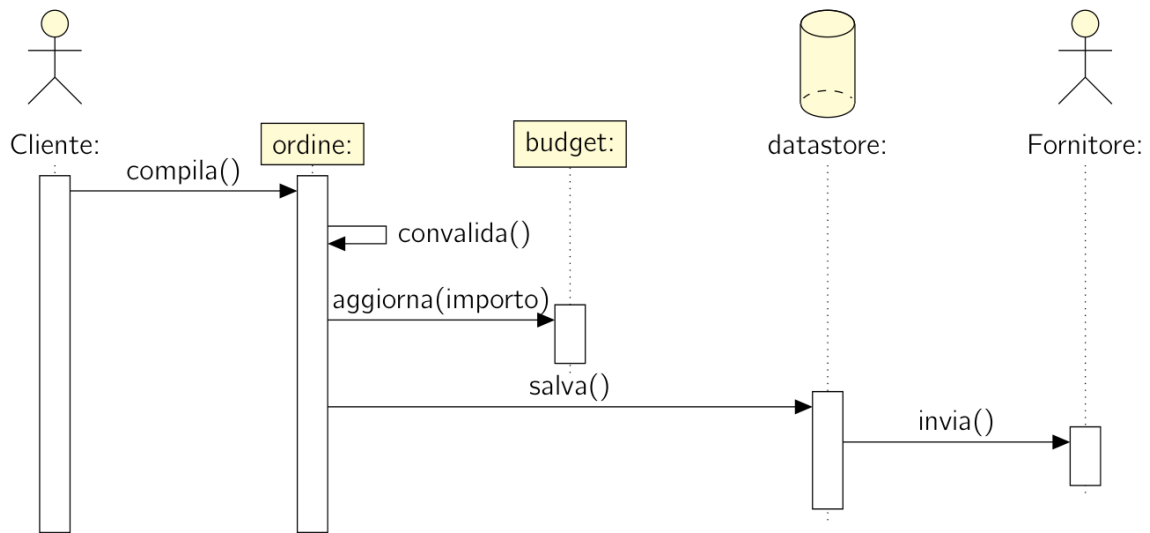
Oppure possiamo rappresentarli tramite diagrammi di sequenza: l'elaborazione sequenziale può essere rappresentata utilizzando invio messaggi solo da sinistra verso destra.

I modelli guidati dagli eventi mostrano come il sistema risponde agli eventi interni o esterni. Il sistema deve avere un numero finito di stati e gli eventi possono causare la transizione da uno stato all'altro. Rappresentiamo questi modelli con i diagrammi di stato UML. Il problema della modellazione basata su stati è che gli stati aumentano velocemente, per ovviare a questo problema di sfruttano degli superstati che comprendono più stati separati. In genere questo tipo di diagrammi vengono integrati con una descrizione testuale.

```

1 -STATI-
2 Attesa:
3     Il forno aspetta l'input. Il display mostra l'ora corrente.
4 Meta' potenza
5     La potenza del forno e' impostata a 300 W. Il display visualizza
6     "Meta' potenza".
7 Massima potenza:

```



8 La potenza del forno e' impostata a 600 W. Il display visualizza
 9 "Massima potenza".
 10 Imposta il tempo:
 11 Il tempo di cottura e' impostato al valore immesso dall'utente.
 12 Il display visualizza il tempo di cottura selezionato e viene
 13 aggiornato non appena l'utente ha finito di impostarlo.
 14 Disabilitato:
 15 Le operazioni del forno sono disattivate per sicurezza; la luce
 16 interna e' spenta, il display visualizza "Attesa".
 17 Abilitato:
 18 Le operazioni del forno sono attive; la luce interna e' accesa;
 19 il display visualizza "Pronto".
 20 Operazione:
 21 Il forno e' in funzione; la luce interna e' accesa; il display visua-
 22 lizza il conto alla rovescia del timer. Alla fine della cottura, il
 23 cicalino suona per 5 secondi. La luce del forno e' accesa. Il
 24 display mostra "Fine cottura" mentre il cicalino suona.

1 -Stimoli-
 2 Meta' potenza:
 3 L'utente ha premuto il pulsante di meta' potenza
 4 Massima potenza:
 5 L'utente ha premuto il pulsante di massima potenza
 6 Timer:
 7 L'utente ha premuto uno dei pulsanti del timer
 8 Numero:
 9 L'utente ha premuto un tasto numerico
 10 Porta aperta:
 11 L'interruttore della porta non e' chiuso
 12 Porta chiusa:
 13 L'interruttore della porta e' chiuso
 14 Avvio:
 15 L'utente ha premuto il pulsante Avvio
 16 Annulla:
 17 L'utente ha premuto il pulsante Annulla

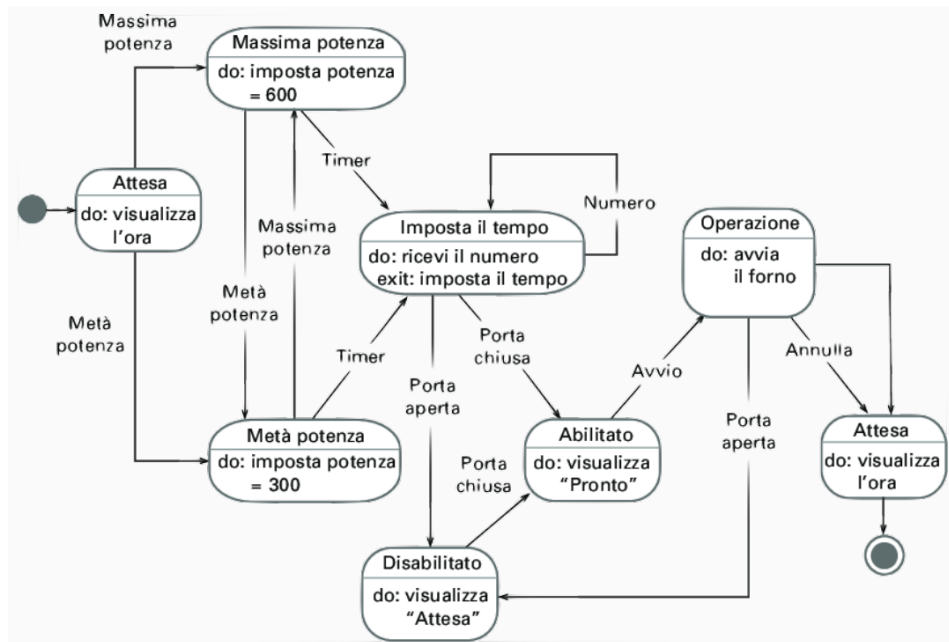


Figura 25: Esempio diagramma di stato (forno a microonde)

7 Progettazione e implementazione

La progettazione e l'implementazione sono fasi del progetto del software in cui viene sviluppato il sistema software eseguibile. La progettazione è un'attività creativa: si occupa delle modalità di risoluzione del problema; si basa sulle richieste del cliente, sulle componenti e sulle loro relazioni. L'implementazione è il processo che realizza il progetto sotto forma di programma.

7.1 Progettazione orientata agli oggetti

Un sistema orientato agli oggetti è formato da oggetti che interagiscono. Gli oggetti mantengono un proprio stato locale (e privato) e forniscono operazioni per modificare tale stato. La progettazione orientata agli oggetti richiede la progettazione di classi e delle loro relazioni. Le classi definiscono gli oggetti nel sistema e le loro interazioni; nel programma eseguibile gli oggetti verranno creati dinamicamente da tali classi.

La progettazione di un sistema da un concetto astratto ad uno dettagliato, orientato agli oggetti, richiede diverse fasi. Vediamo quali:

1. capire e definire contesto ed interazioni esterne con il sistema;
2. progettare l'architettura del sistema;
3. identificare i principali oggetti del sistema;
4. sviluppare i modelli di progettazione;
5. specificare le interfacce.

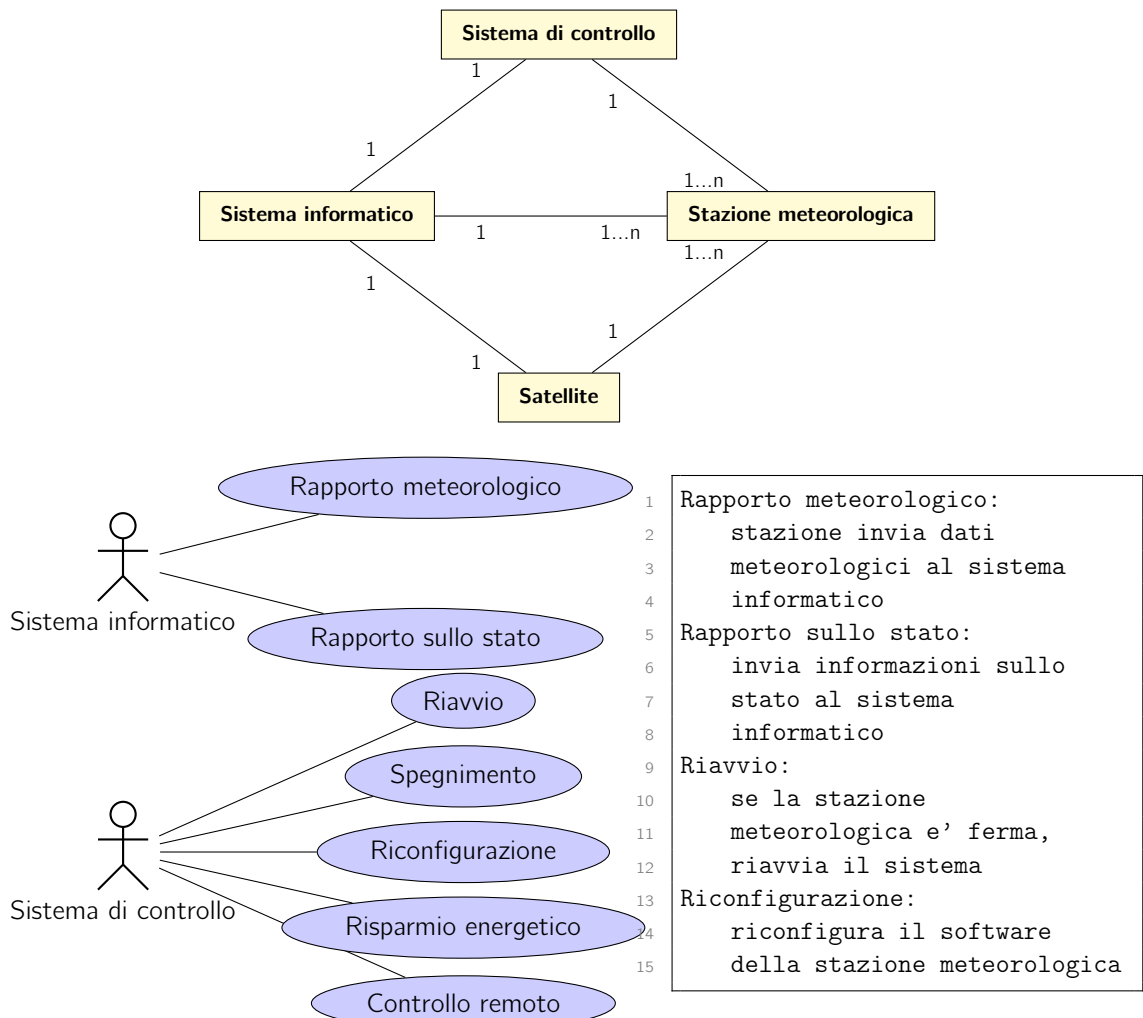
Analizziamo punto per punto prendendo come riferimento l'esempio della stazione meteorologica. Alcune note sull'esempio: stazioni meteorologiche collocate in aree isolate; ciascuna stazione registra informazioni atmosferiche locali; ciascuna stazione invia periodicamente i dati raccolti al sistema informatico mediante collegamento satellitare.

Capire e definire contesto ed interazioni esterne con il sistema Il primo passo è quello di comprendere le relazioni tra software che si sta progettando e ambiente esterno. In questo modo possiamo decidere come fornire le funzionalità richieste per il sistema e come strutturare la comunicazione tra sistema e ambiente. In altre parole, possiamo stabilire i confini. Nel nostro caso di studio, si tratta di decidere come distribuire le funzionalità tra il sistema di controllo di tutte le stazioni ed il software integrato nella singola stazione.

In questa fase è utile utilizzare due modelli:

- modello di contesto: modello strutturale che descrive altri sistemi nell'ambiente in cui si sta sviluppando il sistema;
- modello di interazione: modello dinamico che descrive come sistema interagisce con l'ambiente in fase di utilizzo.

Il modello di contesto è rappresentabile tramite associazioni; sfruttiamo i diagrammi a blocchi. Per modellare interazioni di un sistema con il suo ambiente, utilizziamo un approccio astratto senza specificare troppi dettagli. Utilizziamo dunque i diagrammi per i casi d'uso.



```

1 Sistema:
2   Stazione meteorologica.
3 Caso d'uso:
4   Rapporto meteorologico.
5 Attori:
6   Sistema informatico per la raccolta e l'elaborazione dei dati meteoro-
  
```

logici, stazione meteorologica.

Dati:

La stazione meteorologica invia una sintesi dei dati meteorologici rilevati dagli strumenti nel periodo di rilevazione al sistema informatico. I dati inviati sono le temperature massime, minime e medie del suolo e dell'aria; le pressioni massime, minime e medie dell'aria; le velocità massime minime e medie del vento; le precipitazioni; la direzione del vento, campionata a intervalli di cinque minuti.

Stimolo:

Il sistema informatico stabilisce un collegamento satellitare con la stazione meteorologica e richiede la trasmissione dei dati.

Risposta:

La sintesi dei dati è inviata al sistema informatico.

Commenti:

Alle stazioni meteorologiche viene solitamente chiesto di fare rapporto una volta ogni ora, ma questa frequenza può cambiare da una stazione all'altra e può essere modificata in futuro.

Progettare l'architettura del sistema Nel secondo passo si cerca di combinare

- conoscenza interazioni tra sistema software e ambiente del sistema;
- conoscenze relative alla progettazione architettonica;
- conoscenze relative al dominio di applicazione.

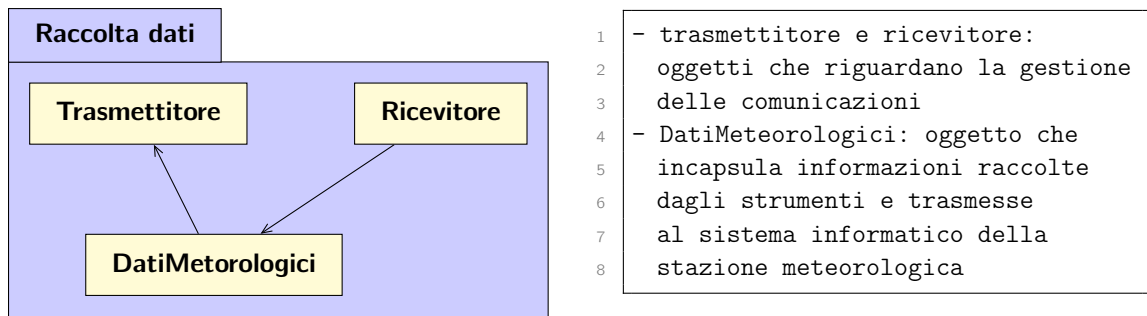
per svolgere le seguenti attività:

- identificare principali componenti che costituiscono il sistema;
- identificare interazioni tra componenti;
- progettazione organizzazione mediante schema architettonico.

Nell'esempio del caso di studio, potremmo organizzare il software in sottoinsiemi indipendenti che comunicano inviando messaggi mediante un'infrastruttura comune, per esempio un Link di comunicazione. In questo modo, ciascun sottosistema, ascolta i messaggi sul Link e raccoglie quelli che lo riguardano.



Il vantaggio è la possibilità di supportare diverse configurazioni di sottoinsiemi, inoltre ogni trasmettitore di messaggio non ha bisogno di indirizzarlo ad un particolare sottosistema ma lo invia direttamente sul Link di comunicazione.



Identificare i principali oggetti del sistema Il terzo passo riguarda l'identificazione delle classi di oggetti. I due passi precedenti dovrebbero già darci un'idea delle classi che necessitiamo; i casi d'uso aiutano per esempio ad identificare alcuni oggetti e le loro interazioni.

Per esempio, nel primo punto del nostro caso di studio, abbiamo definito in un caso d'uso il *Rapporto meteorologico*. Da questo possiamo prendere spunto per identificare alcune classi:

- oggetti che rappresentano gli strumenti di raccolta dati meteo;
- oggetto che rappresenta la sintesi dei dati meteo;
- oggetti ad alto livello che incapsulano interazioni del sistema definite dai casi d'uso.

Esistono delle tecniche di identificazione delle classi di oggetti per i sistemi orientati agli oggetti. Per esempio:

- analisi grammaticale di una descrizione in linguaggio naturale del sistema da costruire: oggetti ed attributi sono nomi, le operazioni o servizi verbi;
- utilizzo entità tangibili (cose) del dominio di applicazione;
- utilizzo analisi basta sugli scenari: analisi di uno scenario alla volta, per ciascuno, il team deve identificare oggetti, attributi ed operazioni.

Nell'esempio del nostro caso di studio, possiamo identificare le classi basandoci sugli elementi hardware della stazione meteorologica e sfruttando la descrizione del sistema e dello scenario (caso d'uso).

StazioneMeteorologica	DatiMeteorologici
identificatore	temperatureAria temperatureSuolo veolcitaVento direzioniVento pressioni precipitazioni
rapportoMeteorologico() rapportoStato() risparmioEnergetico(strumenti) controlloRemoto(comandi) riconfigurazione(comandi) riavvio(strumenti) spegnimento(strumenti)	raccogli() sintetizza()

TermometroSuolo	Anemometro	Barometro
tsIdent temperatura	anIdent velocitaVento direzioneVento	barIdent pressione altezza
acquisisci() test()	acquisisci() test()	acquisisci() test()

```

1 classe StazioneMeteorologica:
2     - interfaccia di base della stazione con il suo ambiente
3     - operazioni basate su interazioni del caso d'uso
4     - singola classe per incapsulare tutte le interazioni
5 classe DatiMeteorologici:
6     - elabora il comando che genera rapporto meteorologico
7     - invia dati di sintesi dagli strumenti alla stazione meteorologica
8 classi TermometroSuolo, Anemometro, Barometro:
9     - in relazione con gli strumenti
10    - riflettono entita' hardware tangibili ed operazioni per controllo hardware
11    - operano in modo autonomo per raccolta e memorizzazione locale dati
12    - dati consegnati ad oggetto DatiMeteorologici in seguito a richiesta

```

In questa fase l'importante è individuare gli oggetti, non tanto la loro implementazione. La progettazione degli oggetti può essere affinata dopo la loro identificazione.

Sviluppare modelli di progettazione I modelli di progettazione o di sistema, mostrano gli oggetti di classe e le loro associazioni, non devono essere troppo astratti (né troppo dettagliati), devono fornire dettagli sufficienti per permettere ai programmatori di fare scelte implementative. In generale, il livello di dettaglio dipende dal processo di progettazione.

Le decisioni da prendere in questo passo sono: scegliere il modello di progettazione; decidere il livello di dettaglio. I due tipi di modelli di progettazione sono:

- modelli strutturali: descrivono struttura statica del progetto tramite oggetti e loro relazioni (generalizzazione, usa/usato da, di composizione);
- modelli dinamici: descrivono struttura dinamica del sistema e mostrano le interazioni tra oggetti del sistema (sequenze di richieste fatte da oggetti, stati del sistema innescati da interazioni tra oggetti).

Per esempio, alcuni modelli che possiamo utilizzare, sono:

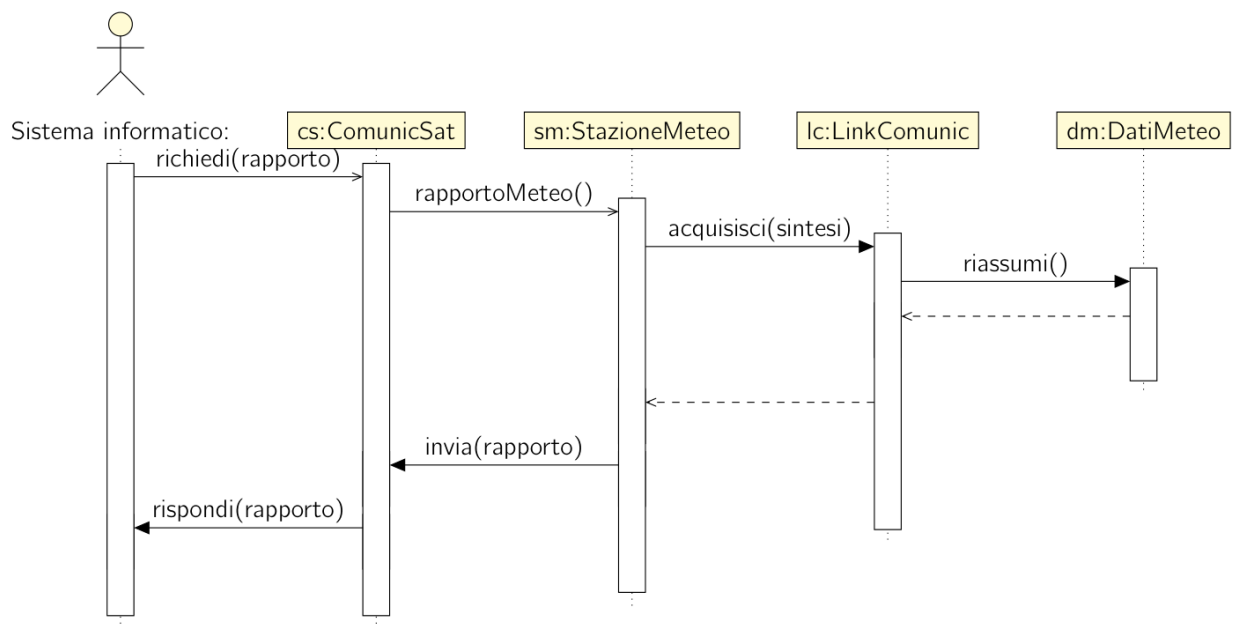
- modelli di sottosistema: raggruppamento logico degli oggetti in sottosistemi coerenti, sono modelli strutturali, sono rappresentati con diagrammi di classe (package) UML;
- modelli di sequenza: mostrano sequenza di interazioni tra gli oggetti, sono modelli dinamici, rappresentati con diagrammi di sequenza UML;
- modelli di macchine a stati: mostrano come oggetti cambiano stato in risposta ad eventi, sono modelli dinamici, sono rappresentati con diagrammi di stato UML.

Solitamente si produce un modello per ciascuna interazione significativa. Se sono presenti modelli di casi d'uso, si crea un modello di sequenza per ciascuno.

```

1 1. oggetto ComunicSat riceve una richiesta dal sistema informatico per
2   avere un rapporto da una stazione meteorologica e accetta richiesta
3 2. ComunicSat invia un messaggio alla StazioneMeteorologica,
4   tramite un collegamento satellitare, per creare una sintesi dei dati raccolti.
5 3. StazioneMeteorologica invia un messaggio all'oggetto
6   LinkComunic per riassumere i dati.
7 4. LinkComunic chiama il metodo di sintesi dell'oggetto
8   DatiMeteorologici e resta in attesa di una risposta
9 5. Elaborazione della sintesi dei dati meteorologici; risultato viene inviato
10 alla StazioneMeteorologica tramite l'oggetto LinkComunic.

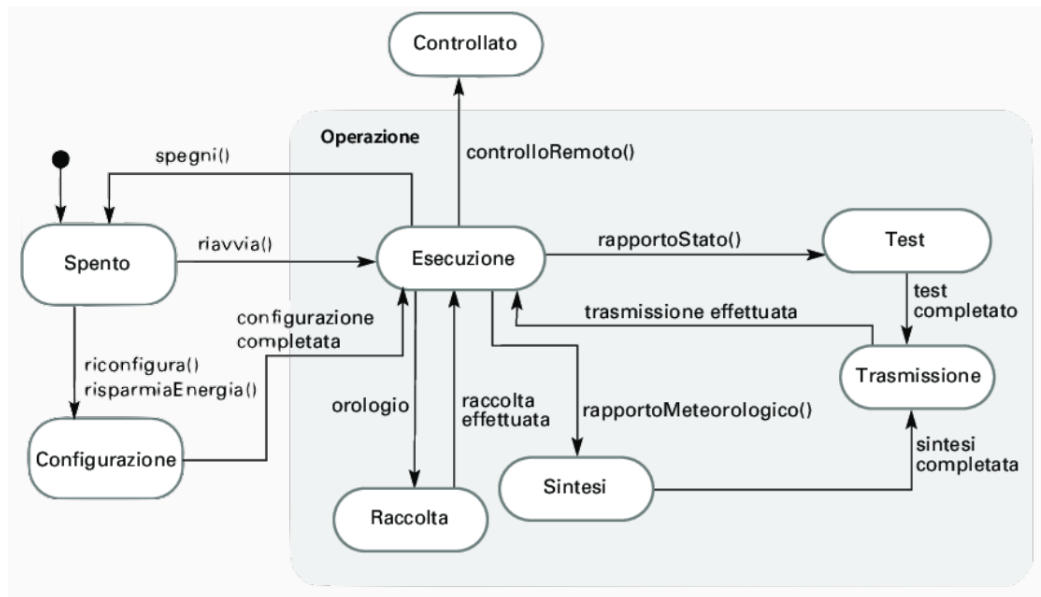
```



- 11 6. StazioneMeteorologica chiama l'oggetto ComunicSat per
 12 trasmettere i dati di sintesi al sistema informatico, tramite satellite.

Nel diagramma di sequenza, la freccia piena indica che oggetto rimane in attesa di risposta; freccia non piena che può fare altro mentre attende. In questo modo, per esempio, ComunicSat e StazioneMeteorologica possono essere implementati come processi concorrenti. Si possono utilizzare diagrammi di stato per sintetizzare il comportamento di un oggetto o di un sottosistema come risposta a messaggi o eventi.

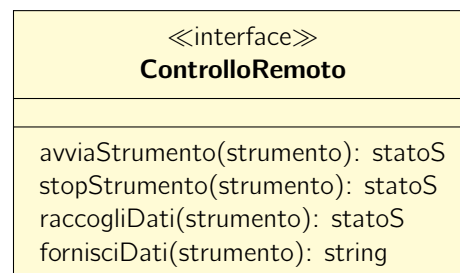
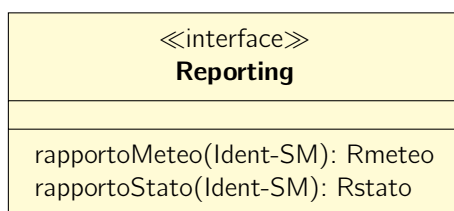
- 1 1. se lo stato del sistema e' Spento, il sistema puo' rispondere a un messaggio
 2 riavvia(), riconfigura() o risparmiaEnergia(). Un messaggio riavvia() provoca
 3 la transizione al normale funzionamento.
 4 Entrambi i messaggi riconfigura() e risparmiaEnergia() provocano la
 5 transizione a uno stato in cui il sistema si autoconfigura.
 6 2. nello stato Esecuzione, il sistema attende ulteriori messaggi.
 7 Se riceve il messaggio spegni(), l'oggetto ritorna allo stato Spento;
 8 3. se riceve il messaggio rapportoMeteorologico(), il sistema passa allo stato
 9 Sintesi. Quando la sintesi e' completata, il sistema passa allo stato
 10 Trasmissione, dove le informazioni vengono trasmesse al sistema remoto;
 11 poi ritorna allo stato Esecuzione;
 12 4. se riceve un segnale dall'orologio, il sistema passa allo stato Raccolta, dove



13 raccoglie i dati dagli strumenti. Ogni strumento, a sua volta, e' programmato
 14 per raccogliere i dati dai suoi sensori;
 15 5. se riceve un messaggio controlloRemoto(), il sistema passa allo stato
 16 Controllato, in cui risponde a diversi gruppi di messaggi dal sistema
 17 di controllo remoto.

Non è necessario creare un diagramma per ogni oggetto, gli oggetti più semplici possono essere descritti senza modello di stato.

Specifica delle interfacce Per specificare le interfacce utilizziamo i diagrammi di classi UML con stereotipo *interface*, in particolare specifichiamo soltanto le operazioni per accedere ad aggiornare i dati.



1 Interfaccia di reporting:
 2 nomi delle operazioni utilizzate per generare i rapporti sulle condizioni
 3 meteo e sullo stato.
 4 Interfaccia di controllo remoto:
 5 quattro operazioni, che si mappano con un singolo metodo nell'oggetto
 6 StazioneMeteorologica, il metodo controlloRemoto.

8 GRASP Patterns

8.1 Progettazione guidata dalle responsabilità

Esiste una differenza tra analisi e progettazione orientata agli oggetti (OOA/D). L'analisi enfatizza l'investigazione di un problema e dei requisiti, piuttosto che una soluzione. D'altro canto, la progettazione si concentra sulla soluzione concettuale che rispetta i requisiti. Per riassumere "Analysis and design have been summarized in the phase do the right thing (analysis), and do the thing right (design)".

Nello specifico l'analisi orientata agli oggetti, si concentra sul trovare e descrivere oggetti nel dominio del problema; la progettazione orientata agli oggetti si concentra sul definire gli oggetti software e come "collaborano" per soddisfare i requisiti.

L'idea di base è quella di pensare alla progettazione degli oggetti software in termini di responsabilità, ruoli e collaborazioni: si parla di **responsibility-driven design (RDD)**. Spesso la progettazione degli oggetti è definita con una qualche variante del tipo: "dopo aver identificato i requisiti e creato il modello di dominio, aggiungere i metodi alle classi e definire i messaggi tra oggetti per soddisfare i requisiti". Si tratta di un'indicazione vaga su come procedere. Risulta cruciale decidere quali metodi e a chi appartengono e come gli oggetti dovrebbero interagire.

Nella prossima sezione tratteremo 9 principi fondamentali nella progettazione degli oggetti e nell'assegnazione delle responsabilità. Questi principi sono noti con il nome di GRASP Pattern. Le responsabilità di un oggetto sono legate agli obblighi di un oggetto in termini di comportamento; distinguiamo due tipi di responsabilità:

- sapere o conoscere (*knowing*): conoscenza di dati privati incapsulati, di oggetti collegati o di cose che possono essere derivate o calcolate;
- fare (*doing*): fare qualcosa a se stesso (es: creare un oggetto o fare un calcolo), iniziare un'azione in altri oggetti oppure controllare e coordinare attività in altri oggetti.

Le responsabilità vengono assegnate alle classi di oggetti durante la fase di progettazione degli stessi. Nota che in generale responsabilità $\neq$ metodo: i metodi sono implementati per affidare o realizzare la responsabilità; le responsabilità sono implementate usando metodi che possono agire individualmente o collaborare con altri metodi ed oggetti. Per esempio la responsabilità di "fornire l'accesso ad una base di dati" potrebbe richiedere decine di classi e centinaia di metodi.

8.2 I GRASP patterns

GRASP sta per General Responsibility Assignment Software Patterns. I GRASP Pattern aiutano a comprendere gli aspetti essenziali del OOD (analisi e progettazione orientata agli oggetti) e ad applicare i ragionamenti relativi alla fase di progettazione in maniera metodica, razionale, e spiegabile. L'elenco dei GRASP pattern è il seguente:

1. Creator;
2. Information Expert;
3. Low coupling;
4. Controller;
5. High Cohesion;
6. Polymorphism;

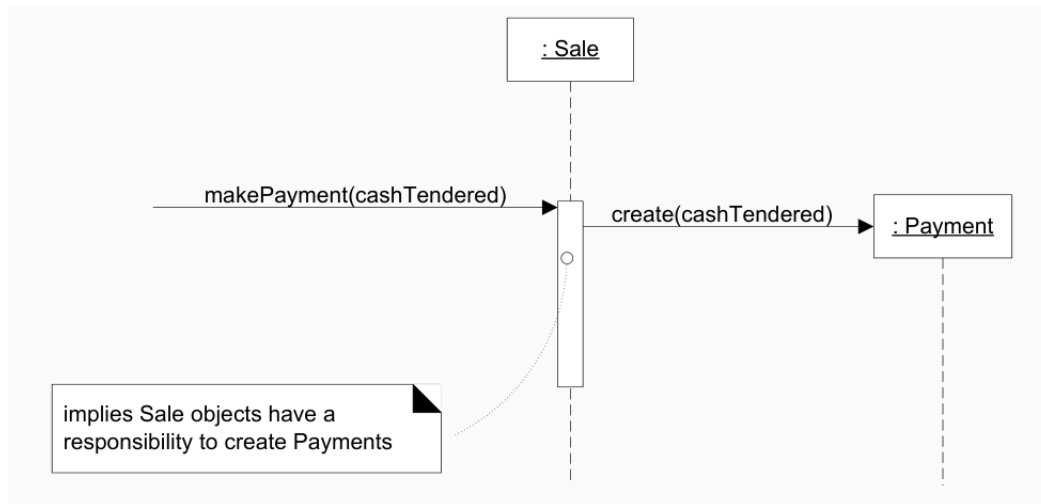


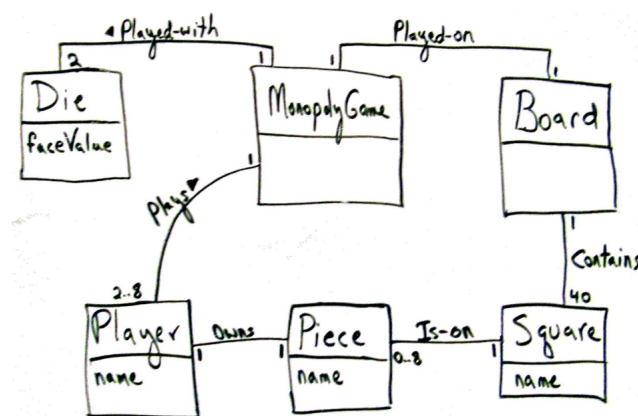
Figura 26: Esempio: Sale ha la responsabilità di creare Payment

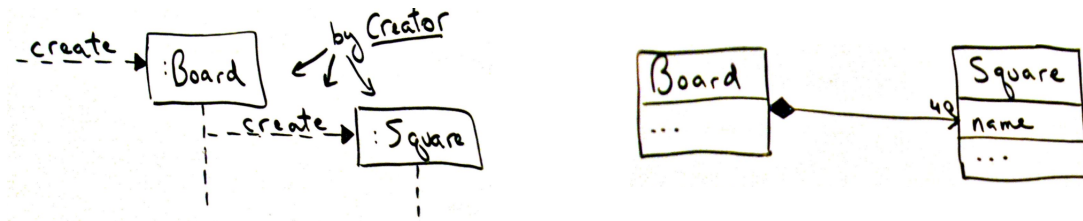
7. Pure Fabrication;
8. Indirection;
9. Protected Variations.

Creator Creazione di oggetti in sistemi OO è una delle attività più frequenti, il pattern per assegnare la responsabilità di creazione può essere molto utile.

- Problema: chi è responsabile della creazione di una nuova istanza di una classe?
- Soluzione: Assegnare alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera:
 - B “contiene” e aggrega (composizione) A;
 - B registra (record) A;
 - B usa A;
 - B ha i dati per inizializzare A, dati che passa ad A quando viene creato. Come vedremo dopo, B è un *Expert* rispetto alla creazione di A.

Se più di una delle condizioni è applicabile, si preferisce aggrega o contiene.

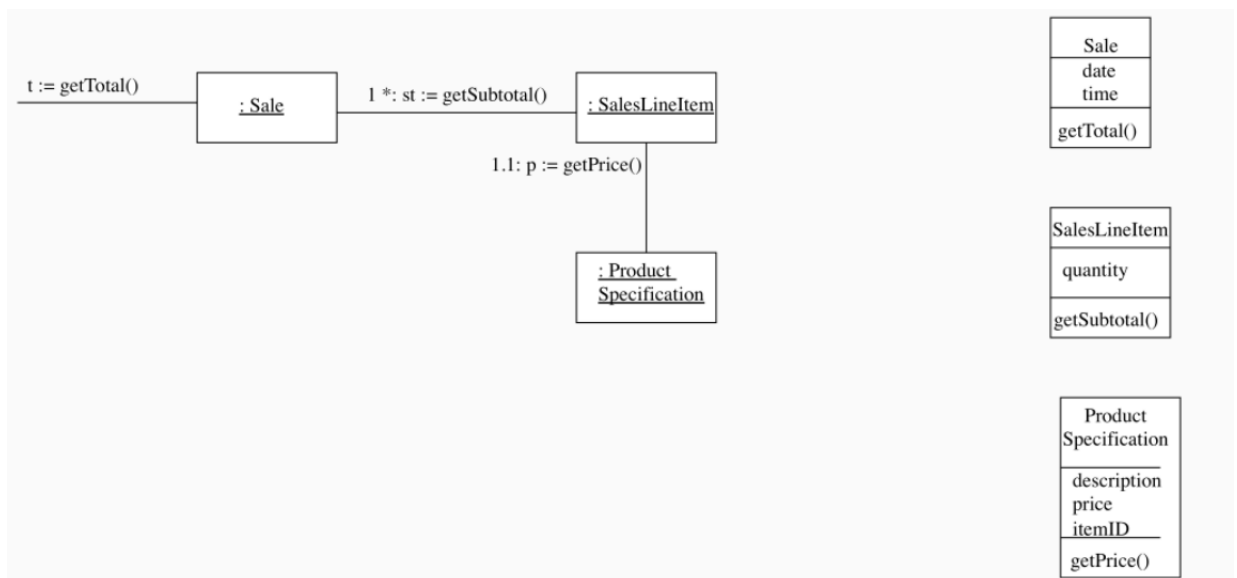




Information Expert

- Problema: Qual è il principio generale per assegnare responsabilità ad un oggetto?
- Soluzione: Assegnare la responsabilità all'esperto (information expert), la classe che ha l'informazione necessaria per adempiere al compito/responsabilità.

Esprime la comune intuizione che un oggetto fa cose in relazione con informazioni che possiede. La domanda è: dove effettuiamo la ricerca della classe che ha l'informazione? Nel modello di dominio o nel modello di progettazione? Se ci sono classi rilevanti nel modello di progettazione, optiamo per quello; altrimenti, guardiamo a livello di dominio e ci facciamo guidare nella creazione delle classi (a livello di progettazione).



Classe	Responsabilità
Sale	conosce il totale della vendita
SalesLineItem	conosce il sottototale dell'item
ProductDescription	conosce il prezzo del prodotto

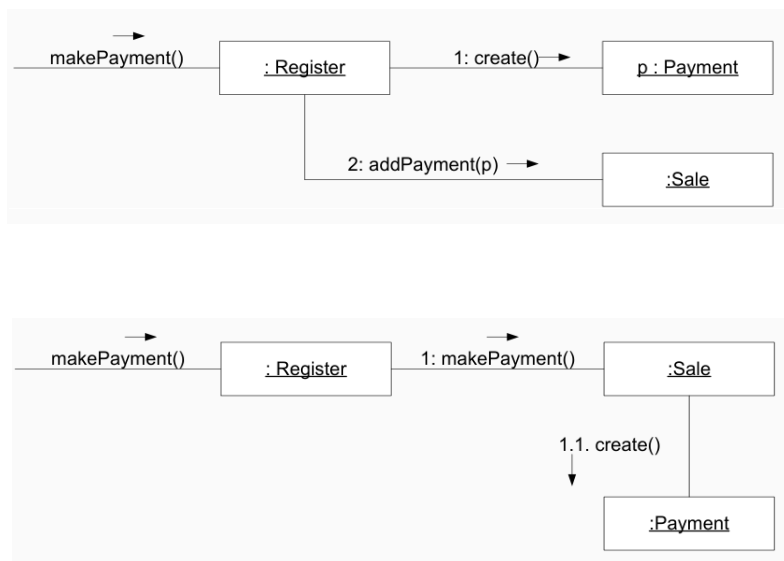
L'adempimento delle responsabilità richiede informazioni "sparse" per diverse classi di oggetti: diversi esperti parziali che devono collaborare (nell'esempio del calcolo del totale, sono 3 classi). Uno dei benefici di tale pattern è quello di mantenere l'incapsulamento.

Talvolta, la soluzione proposta dal pattern non è desiderabile. Nell'esempio riportato, chi è responsabile per salvare l'oggetto Sale nella base di dati? La classe Sale? Ogni classe deve quindi gestire la "propria" memorizzazione? Se utilizziamo Sale a questo scopo, allora la classe non è più focalizzata solo alla logica legata alla vendita. Questo viola il principio architetturale di separazione delle principali "preoccupazioni" del sistema.

Low coupling

- Problema: come supportare basso livello di dipendenza, basso impatto in caso di cambiamenti e alto livello di riutilizzo?
- Soluzione: assegnare responsabilità in modo che l'accoppiamento (coupling) rimanga basso. Utilizzare tale principio per valutare delle alternative.

Il coupling è una misura di quanto un elemento sia connesso, abbia conoscenza di, o si appoggi su altri elementi. Una classe con low coupling non è dipendente da molte altre classi. In certi casi un alto (o forte) accoppiamento non è desiderabile in quanto può cambiamenti nella classe possono provocare una catena di cambiamenti forzati alle classi collegate, risulta difficile comprendere la classe quando considerata da sola e risulta anche difficile da riutilizzare in quanto richiede la presenza delle altre da cui dipende (caso non indipendente).

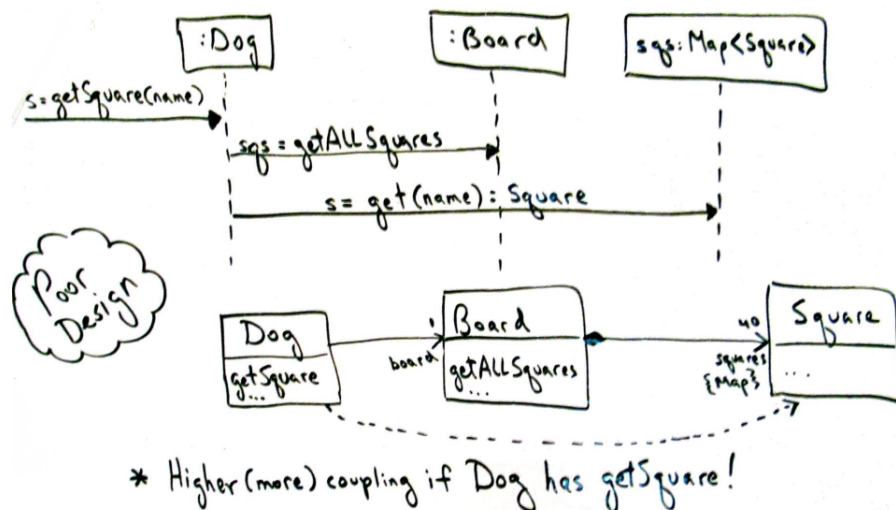


Supponiamo di dover creare un'istanza di `Payment` e doverla associare a `Sale`. Quale classe è responsabile di questo compito? `Register` registra i pagamenti, quindi il pattern Creator suggerisce `Register`; quest'ultimo potrebbe inviare un pagamento a `Sale` con l'istanza di `Payment` come parametro. Tuttavia, il pattern Low coupling, suggerisce la seconda soluzione in quanto il livello di accoppiamenti è più basso.

I benefici di tale pattern sono:

- comprensibilità: le classi sono generalmente più semplici da comprendere quando isolate;
- manutenibilità: classi non sono influenzate dai cambiamenti di altri componenti;
- riusabilità: classi facili da riutilizzare senza grosse dipendenze.

Controller Consideriamo una semplice struttura a strati composta da un UI layer e da un Domain layer. Definiamo degli attori umani che interagiscono con il gioco. Il gioco che prendiamo in considerazione è il gioco del Monopoly (digitale), in cui si interagisce via click con il mouse; gli oggetti software della UI reagiscono al click del mouse e fanno iniziare il gioco. Se seguiamo il principio Model-View (Sezione 5.2.1), sappiamo che gli oggetti della UI non devono contenere business logic, quindi si occuperanno di catturare l'evento (click) e di delegare altri oggetti. Il pattern controller risponde alla domanda: qual è il primo oggetto



dopo/oltre lo strato UI che dovrebbe ricevere il messaggio dallo strato UI? Come connettere lo UI layer a quello della logica dell'applicazione?

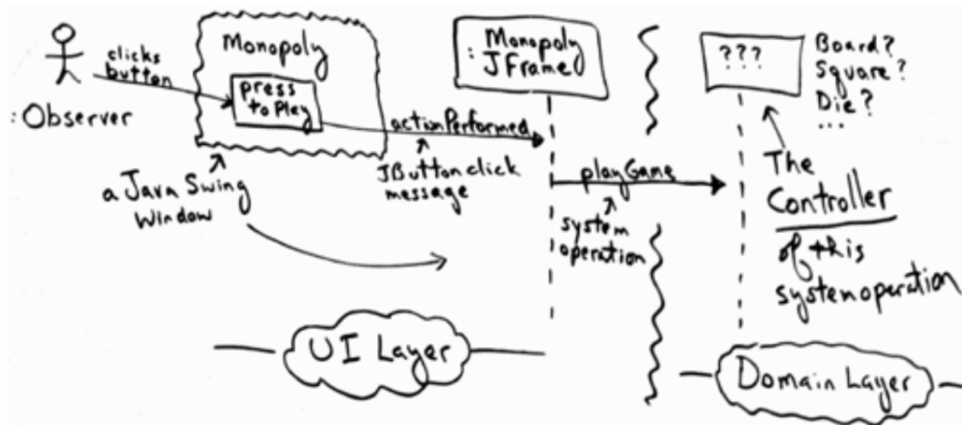
- Problema: qual è il primo oggetto dopo lo strato UI che riceve e coordina ("controlla") un'operazione del sistema?
- Soluzione: Assegnare la responsabilità ad un oggetto che rappresenta una delle seguenti scelte:
 - rappresenta il sistema nella sua totalità, un "root object", uno strumento all'interno del quale il software è in esecuzione o un sottosistema. Si parla di *facade controller*;
 - rappresenta lo scenario di un caso d'uso in cui l'operazione del sistema ha luogo; il nome associato è generalmente <UseCaseName>Handler, <UseCaseName>Coordinator, o <UseCaseName>Session. Si parla di *session controller*.

Con "operazioni di sistema" si intendono i principali eventi di input di sistema, per esempio: cassiere preme il bottone End Sale e viene generato l'evento "la vendita è terminata". Il Controller è il primo oggetto oltre lo strato di UI responsabile per ricevere o gestire un messaggio legato ad una operazione di sistema.

Nell'esempio del gioco del Monopoly, è la board il primo oggetto a ricevere il messaggio playGame?

Possiamo considerare una delle seguenti opzioni: rappresentare l'intero sistema o un root object (chiameremo tale oggetto MonopolyGame); rappresentare lo strumento (hardware) su cui il software è in esecuzione (es: un telefono) o un dispositivo specifico della banca; rappresentare un caso d'uso (chiameremo tale caso d'uso Play Monopoly Game e considereremo una classe PlayMonopolyGameHandler).

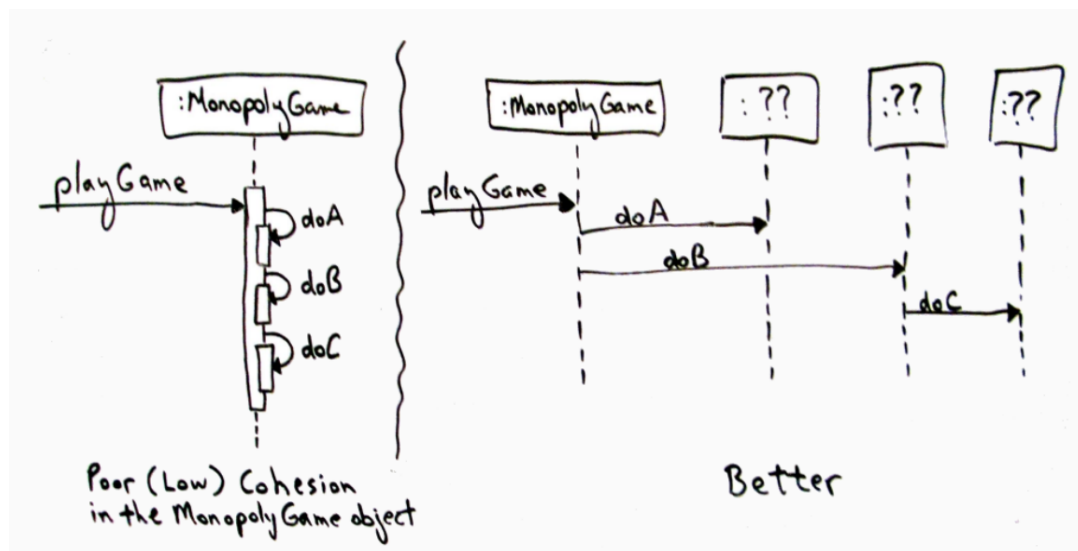
Normalmente un controller dovrebbe delegare ad altri oggetti il lavoro che deve essere fatto, esso deve occuparsi solamente di coordinare e controllare le attività, non si occupa esso stesso di gran parte del lavoro. Una situazione che vogliamo evitare è il **bolated controller**: si tratta di un controller non focalizzato che gestisce troppe aree di responsabilità. Vedremo che si tratta di un controller a bassa coesione. Tale situazione si verifica quando c'è una singola classe controller che riceve/deve gestire tutti gli eventi di sistema (e sono tanti), o se si occupa di molte attività, senza delegarne alcuna.



High Cohesion

- Problema: Come mantenere gli oggetti focalizzati, comprensibili, gestibili e come side effect supportare Low Coupling?
- Soluzione: Assegnare una responsabilità in modo che la coesione rimanga elevata. Utilizzato per valutare alternative.

In termini di progettazione degli oggetti, con il termine functional cohesion si intende una misura del grado in cui le responsabilità di un elemento sono fortemente correlate e focalizzate. Una classe con bassa coesione fa molte cose, scollegate tra loro. Si tratta di una condizione non desiderabile: la classe risulta difficile da comprendere, da utilizzare e da mantenere. Rule of thumb: una classe ad alta coesione ha un numero relativamente piccolo di metodi, con funzionalità correlate e collabora con altri oggetti per lo svolgimento di task complessi.



Polymorphism

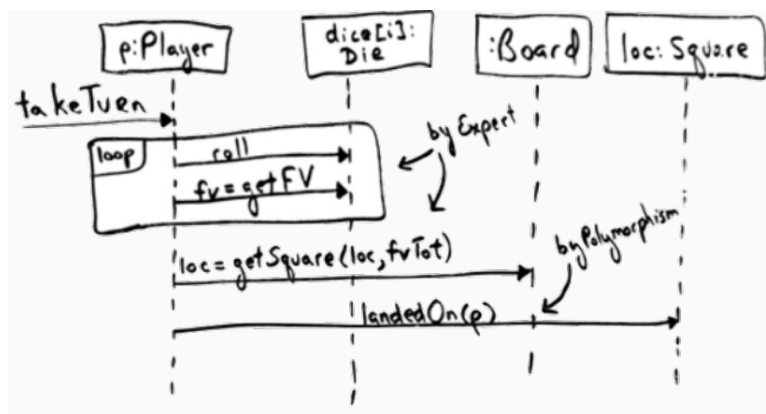
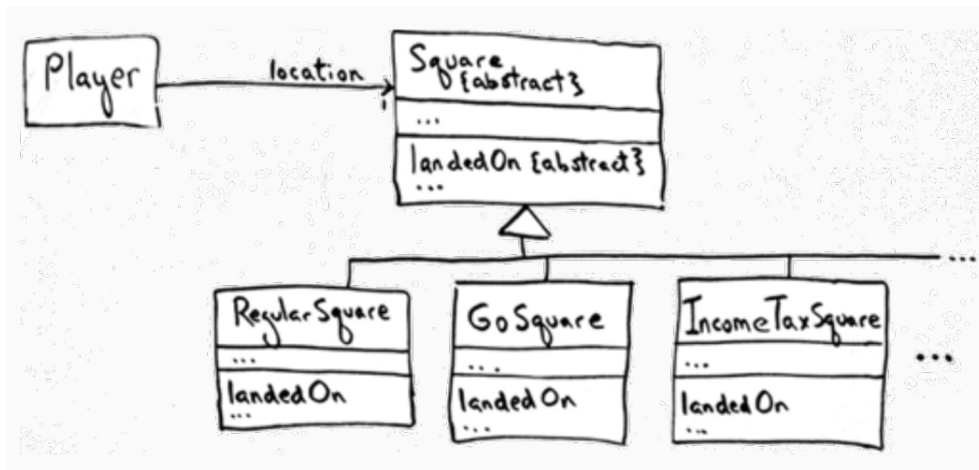
- Problema: come gestire alternative basate sul tipo? Come creare componenti software collegabili (pluggable)?

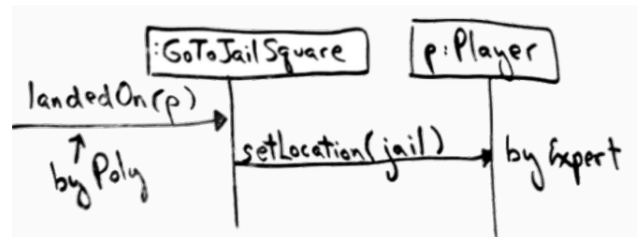
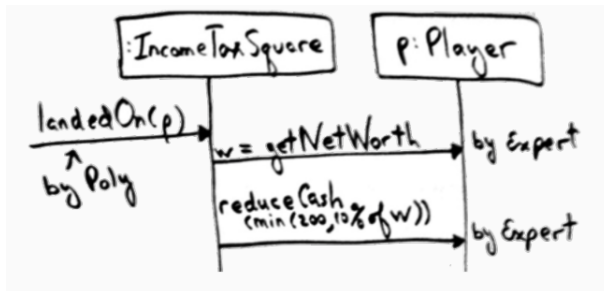
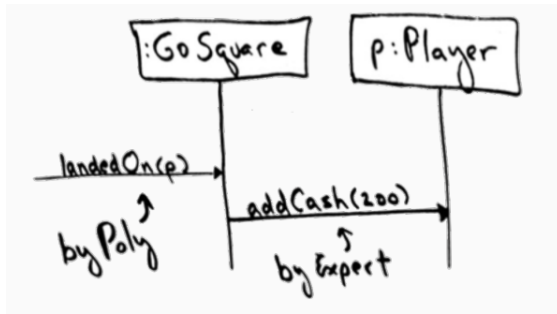
- Soluzione: quando alternative o comportamenti variano da tipo a tipo (classe), assegnare la responsabilità relative al comportamento – usando operazioni polimorfiche – ai tipi per cui il comportamento varia. Corollario: non testare per il tipo di un oggetto e utilizzare la logica condizionale per eseguire le varie alternative basate sul tipo.

Cosa intendiamo per pluggable software components? Consideriamo un sistema client-server: come possiamo sostituire un componente nel server senza influire sul client?

Nell'esempio del NextGen POS, dobbiamo interfacciarci con molteplici moduli (esterni) per il calcolo delle tasse (es: Tax-Master, Good-As-Gold Tax Pro) e il nostro sistema deve essere in grado di integrarne diversi. Siccome ciascun modulo ha un'interfaccia simile ma diversa, dobbiamo identificare un oggetto responsabile per la gestione dei vari calcolatori di tasse. Una possibile soluzione è quella di assegnare la responsabilità di adattamento ai singoli oggetti corrispondenti ai diversi calcolatori di tasse (o adattatori), implementati con una operazione `getTaxes` polimorfica che verrà realizzata in maniera differente per ogni calcolatore. Gli adattatori sono degli oggetti locali che rappresentano i calcolatori di tasse esterni, non sono parte di o gli stessi calcolatori esterni.

Nell'esempio del Monopoly, quando il giocatore arriva (lands on) in una particolare cella avviene un'azione specifica (es: la casella di via dona 200\$). Sfruttando il principio del polimorfismo possiamo creare un'operazione polimorfica per ciascun tipo di casella il cui comportamento varia. Dal momento che il comportamento è legato all'arrivare su una casella, un buon nome per l'operazione è `landsOn`



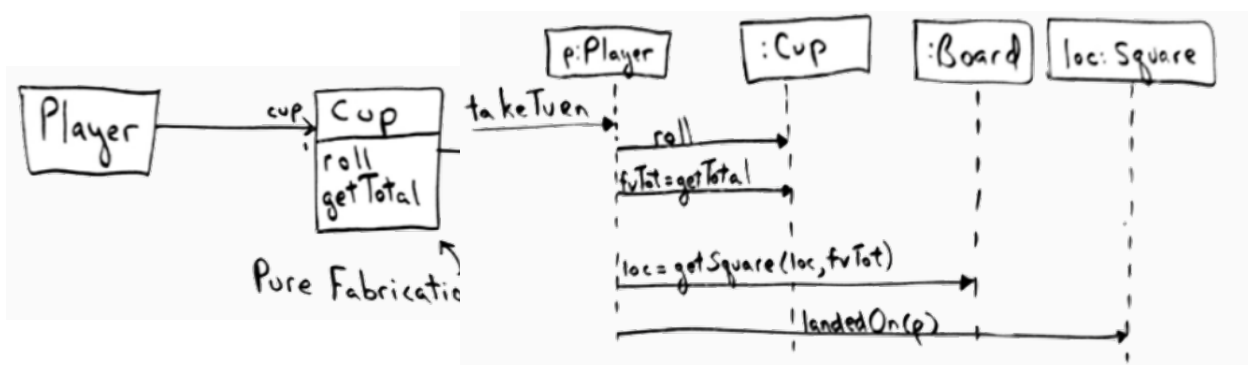


Pure Fabrication

- Problema: quale oggetto deve avere la responsabilità quando non vogliamo violare i principi di High Cohesion e Low Coupling (o altri obiettivi) ma la soluzione offerta dal pattern Expert (o altri) non è appropriata?
- Soluzione: assegnare un insieme altamente coeso di responsabilità ad una classe "artificiale", che non rappresenta un concetto nel dominio.

In molte situazioni, considerare solo le classi "del dominio", può portare a decisioni non appropriate in termini di riuso, coesione e/o accoppiamento. Il pattern in questione ci viene in aiuto.

Nell'esempio del Monopoly, possiamo creare una classe Cup che si occupi del lancio dei dadi e di comunicare al Player la somma delle facce.



Indirection Pattern

- Problema: come assegnare una responsabilità in modo da evitare un accoppiamento diretto tra due o più cose? Come disaccoppiare gli oggetti così da ottenere un basso livello di accoppiamento e aumentare la possibilità di riutilizzo?
- Soluzione: assegnare la responsabilità ad un oggetto intermedio che funga da mediatore tra altri componenti o servizi, così che questi non siano direttamente accoppiati.

Per esempio, nell'esempio del NextGen POS, possiamo definire l'oggetto `TaxCalculatorAdapter`: mediante polimorfismo gli Adapter aiutano a fornire interfaccia comune verso diversi servizi per il calcolo delle tasse. Attraverso l'Adapter ci proteggiamo da eventuali cambiamenti interni di servizio che non possiamo controllare. Sempre nell'esempio del NextGen POS, possiamo definire `PersistentStorage`: in questo caso si sfrutta il Pure Fabrication: disaccoppiamo `Sale` dal salvataggio dell'oggetto; `PersistentStorage` agisce come intermediario tra `Sale` ed il database.

Protected Variations

- Problema: come progettare oggetti, sottosistemi, e sistemi così che variazioni o instabilità in questi elementi non abbiano un impatto indesiderabile su altri elementi?
- Soluzione: identificare i punti di possibile futura variazione o instabilità; assegnare le responsabilità in modo da creare un'interfaccia stabile attorno ad essi

Nel caso del Tax Calculator, un possibile punto di instabilità può essere dato dall'interazione con differenti interfacce o API dei servizi esterni per il calcolo delle tasse. Il sistema NextGen POS deve essere in grado di integrare diversi servizi per il calcolo delle tasse e, in futuro, altri servizi di terze parti non ancora esistenti. Per risolvere tale situazione, possiamo definire un'interfaccia `ITaxCalculatorAdapter` implementata per i diversi servizi. In questo modo aggiungiamo uno strato di protezione (grazie a indirection), inoltre gli oggetti "interni" collaborano con un'interfaccia stabile.

9 GoF Design Patterns

Spesso, durante la progettazione di software, si trovano diverse soluzioni per risolvere un stesso problema e non si sa bene quale scegliere. Altrettante volte capita di trovarsi a risolvere problemi simili ad altri già affrontati e risolti, in questi casi non conviene ripartire da zero, si cercano di sfruttare soluzioni che hanno funzionato in passato. I GoF (*Gang of Four*) Design Patterns cercano di chiarire questo tipo di dubbi fornendo soluzioni a problemi comuni.

Nel libro, i pattern, vengono analizzati sotto diversi punti, tra cui i seguenti:

- intento: breve risposta alla domanda: cosa fa il pattern? A quale particolare problema di progettazione è rivolto?
- struttura: rappresentazione grafica usando notazione Object Modeling Technique (OMT);
- partecipanti: classi e/o oggetti coinvolti e loro responsabilità;
- collaborazioni: come collaborano i partecipanti per assumersi le loro responsabilità;
- pattern correlati: quali pattern sono strettamente correlati, quali sono le differenze, e con quali altri pattern dovrebbe essere usato

I Gof Pattern sono 23 in totale, una loro possibile organizzazione è basata su:

- scopo: ciò che il pattern fa: creazionale, strutturale, comportamentale;
- raggio d'azione: specifica se il pattern si applica principalmente a classi o oggetti: class pattern, object pattern.

I pattern creazionali si occupano di inizializzare e configurare oggetti; i pattern strutturali formano composizioni di classi o oggetti; i pattern comportamentali caratterizzano i modi in cui classi o oggetti interagiscono e distribuiscono le responsabilità.

I class patter riguardano le relazioni tra classi e sottoclassi espresse tramite l'ereditarietà, sono dunque relazioni statiche, definite a tempo di compilazione; gli object pattern riguardano relazioni tra oggetti, possono essere cambiate in fase di esecuzione e sono più dinamiche.

Combinando scopo e raggio d'azione si ottiene:

- Pattern creazionali
 - class pattern: delegano parte del processo di creazione di un oggetto alle sotto-classi;
 - object pattern: delegano parte del processo di creazione ad altri oggetti;
- Pattern strutturali
 - class pattern: usano l'ereditarietà per comporre le classi;
 - object pattern: descrivono modi per raggruppare gli oggetti;
- Pattern comportamentali
 - class pattern: usano l'ereditarietà per descrivere algoritmi e flussi di controllo;
 - object pattern: descrivono il modo in cui gruppi di oggetti cooperano per svolgere un'attività che un singolo oggetto non potrebbe svolgere.

		Scopo		
		Creazionale	Strutturale	Comportamentale
RdA	Classi	Factory Method	Adapter (class)	Interpreter Template Method
	Oggetti	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Figura 27: Organizzazione dei design patterns

9.1 Il catalogo dei pattern

Per vedere esempi di implementazione per ogni pattern: <https://github.com/iluwatar/java-design-patterns>

Pattern creazionali

- Singleton: fa in modo che una classe abbia una ed una sola istanza, e fornisce un punto di accesso globale a questa.

- Simple Factory: Crea oggetti specializzati e complessi.
- Abstract Factory: Fornisce un'interfaccia per la creazione di famiglie di oggetti correlati o dipendenti senza specificare e conoscere le classi concrete.
- Factory Method: Definisce un'interfaccia per creare un oggetto ma lascia decidere alle sottoclassi quale oggetto istanziare.
- Builder: Separa la costruzione di un oggetto complesso dalla sua rappresentazione, così che lo stesso processo di costruzione possa creare rappresentazioni diverse.
- Prototype: Specifica gli oggetti da creare utilizzando un'istanza prototipale; la creazione dei nuovi oggetti avviene replicando il prototipo.
- Lazy initialization: Ritarda una creazione "costosa" fino a quando non è necessaria.

Pattern strutturali

- Adapter: Converte l'interfaccia di una classe in un'altra interfaccia richiesta dal client.
- Bridge: Disaccoppia un'astrazione dalla sua implementazione in modo che le due possano variare indipendentemente.
- Composite: Raggruppa gli oggetti in strutture ad albero che rappresentano gerarchie di tipo parti-tutto. Permette al client di trattare oggetti singoli e raggruppamenti in modo uniforme.
- Decorator: Aggiunge dinamicamente responsabilità a un oggetto. Alternativa flessibile all'ereditarietà per estendere le funzionalità fornite.
- Facade: Definisce un'interfaccia singola (a più alto livello) a un insieme interfacce di un sottosistema.
- Flyweight: Usa la condivisione per gestire efficacemente numerosi oggetti a granularità fine.
- Proxy: Fornisce un surrogato di un oggetto per controllare l'accesso a quest'ultimo.

Pattern comportamentali

- Chain of Responsibility: Evita di accoppiare il mittente di una richiesta con il suo destinatario dando la possibilità a più di un oggetto di gestire la richiesta. Si ordinano gli oggetti destinatari e la richiesta passa da oggetto in oggetto finché non viene gestita.
- Command: Incapsula una richiesta di un oggetto, rendendo possibile parametrizzare i client con diverse tipologie di richieste (bufferizzate, registrate, di cui è possibile annullare l'effetto).
- Iterator: Fornisce un modo per accedere agli elementi di un contenitore sequenzialmente, senza esporre la struttura dati sottostante.
- Interpreter: Dato un linguaggio, definisce una rappresentazione della grammatica e dell'interprete del linguaggio stesso.
- Mediator: Definisce un oggetto che incapsula le strategie di collaborazione di un gruppo di oggetti.

- Memento: Senza violare l'incapsulamento, coglie e rende esplicito lo stato interno di un oggetto, così da poterlo portare poi allo stato iniziale.
- Observer: Definisce una dipendenza uno a molti fra oggetti in modo che quando un oggetto cambia stato, tutti gli oggetti che dipendono da questo vengono aggiornati o ricevono notifica.
- State: Permette ad un oggetto di modificare il proprio comportamento quando il suo stato interno cambia.
- Strategy: Definisce una famiglia di algoritmi, li incapsula e li rende interscambiabili tra loro.
- Template Method: Definisce lo scheletro di un algoritmo in un metodo, delegando alcuni passi alle sottoclassi.
- Visitor: Rappresenta un'operazione da svolgersi sugli elementi di una struttura di oggetti. Permette di definire nuove operazioni senza cambiare le classi degli elementi su cui opera.

9.2 I GoF pattern nel dettaglio

In questa sezione ci proponiamo di analizzare alcuni dei GoF pattern.

Adapter Pattern

- Problema: come risolvere l'incompatibilità tra interfacce o fornire un'interfaccia stabile verso componenti simili ma con diverse interfacce?
- Soluzione (consiglio): convertire l'interfaccia originale di un componente in un'altra interfaccia, mediante un oggetto adapter intermedio.

Factory Pattern Viene chiamato Simple Factory o Concrete Factory, non è propriamente un GoF pattern ma è molto diffuso. Si appoggia sul principio di Pure Fabrication factory.

- Problema: chi è responsabile per la creazione di oggetti quando ci sono speciali fattori da considerare, ad esempio una logica complessa di creazione, il desiderio di separare le responsabilità di creazione per una miglior coesione, ...?
- Soluzione (consiglio): creare un Pure Fabrication object chiamato Factory che gestisce la creazione.

Singleton Pattern

- Problema: è ammessa esattamente una istanza della classe; è un singleton. è necessario un singolo punto di accesso globale.
- Soluzione: Definire un metodo statico della classe che restituisce il singleton.

A livello pratico si può creare una variabile statica privata, possibilmente chiamata `theInstance` o `instance`, che memorizza l'istanza. Quindi si definisce un metodo pubblico statico, possibilmente chiamato `getInstance()`; la prima volta che viene chiamato il metodo, un costruttore privato crea un'istanza e la salva nella variabile statica privata.

Strategy Pattern

- Problema: come ammettere vari, ma affini, algoritmi/policy/strategie? Come progettare la possibilità di modificare tali algoritmi/policy/strategie?
- soluzione (consiglio): Definire ciascun algoritmo/policy/strategia in una classe separata con un'interfaccia comune.

L'oggetto strategy è associato ad un context object, cioè l'oggetto a cui si applica l'algoritmo.

Composite Pattern

- Problema: come trattare un gruppo o una composizione di oggetti allo stesso modo di un oggetto atomico (non composto)?
- Soluzione (consiglio): definire classi per oggetti atomici e composti che implementano la stessa interfaccia.

Facade Pattern

- Problema: è richiesta un'interfaccia comune ed unificata verso un insieme di implementazioni o interfacce eterogenee – ad esempio in un sottosistema. Ci potrebbe essere un accoppiamento non desiderato con molteplici parti del sottosistema, o l'implementazione del sottosistema potrebbe cambiare.
- Soluzione (consiglio): definire un singolo punto di contatto con il sottosistema – un oggetto facade che avvolga (wrap) il sottosistema; tale oggetto presenta un'unica interfaccia unificata ed è responsabile per l'interazione con i componenti del sottosistema.

Un facade è sostanzialmente un oggetto frontend che fornisce un singolo punto di accesso ai servizi di un sottosistema. L'implementazione ed altre componenti del sottosistema sono privati, non accessibili dall'esterno.

Observer Pattern

- Problema: differenti tipi di oggetto subscriber sono interessati a cambiamenti di stato o eventi generati dall'oggetto publisher. Il publisher vuole mantenere low coupling con i subscriber. Come procedere?
- Soluzione (consiglio): definire un'interfaccia "subscriber" (o "listener"). I subscriber implementano tale interfaccia. Il publisher può dinamicamente registrare i subscriber interessati ed inviare loro le notifiche relative ad un cambiamento/evento.

Le notifiche possono essere:

- Push notification: ogni volta che lo stato del soggetto cambia, tutti gli observer ricevono una notifica del cambiamento.
- Push-Update Notification: il soggetto manda anche lo stato che è stato modificato.
- Pull Notification: l'observer richiede lo stato del soggetto.

10 Test del software

L'obiettivo è quello di dimostrare che un programma svolge i compiti per cui è stato realizzato e identificare eventuali errori prima di metterlo in uso. Per raggiungere tale scopo il programma viene eseguito con dati artificiali, attraverso i risultati dei test, identifichiamo eventuali errori, anomalie o informazioni sui requisiti non funzionali del programma.

10.1 Stadi di test

Gli stadi del test sono tipicamente 3.

Test dello sviluppo Comprende tutte le attività di test svolte dal team di sviluppo, tra le quali: test delle unità; test dei componenti; test del sistema. Si tratta sostanzialmente di testare tutte le singole unità di programma, quindi tutte le funzioni/metodi con differenti parametri di input. Spesso questi test vengono automatizzati, vengono utilizzati a questo scopo dei framework che dipendono dal linguaggio utilizzato. In Java viene utilizzato JUnit. I test automatici sono coposti da tre fasi:

1. parte di impostazione: inizializzato il sistema con il test case (input e output previsti);
2. parte della chiamata: chiamata relativa ad oggetto o metodo da testare;
3. parte dell'asserzione: confronto tra risultato della chiamata e risultato previsto. Se l'asserzione è vera, allora il test è avvenuto con successo.

Test della release Viene testata una particolare versione del sistema. La release potrebbe essere utilizzata da clienti, utenti, membri del team che stanno sviluppando sistemi correlati o membri del management. I test che vengono svolti sono test riguardanti requisiti, scenari e prestazioni. Si tratta di parametri fissati inizialmente con il cliente.

Test degli utenti Vengono divisi in tre:

- alpha test: alcuni utenti selezionati operano a fianco del team di sviluppo per testare le prime release del sistema;
- beta test: si effettua su una prima release del sistema (anche incompleta). Tale release viene messa a disposizione di un gran numero di utenti, in questo modo il software viene testato su un gran numero di configurazioni;
- test di accettazione: i clienti testano il sistema con i loro dati e decidono se il sistema deve essere accettato.

